

UNIVERSIDAD DEL BOSQUE
Facultad de Ingeniería
Ingeniería de Software & Ingeniería en Robótica

Competitive Programming Notebook

Algoritmos, estructuras de datos y plantillas

C++

Juan Carlos Morales
Doble titulación: Sistemas & Robótica

Bogotá, Colombia · 2026

Índice

I	Introduccion	5
1	Fundamentos Básicos	5
1.1	Entrada / Salida Rápida (Fast I/O)	5
1.2	Formateo de Salida	5
1.3	Condicionales y Ciclos	6
1.4	Conversiones, Parseos y Casteos	6
II	Estructuras de Datos	7
2	Basic Structures	7
2.1	Matrices (Arreglos 2D)	7
2.2	Vectores / Listas dinámicas	7
2.3	Listas enlazadas (LinkedList)	7
2.4	Pilas (Stack)	7
2.5	Colas (Queue)	7
2.6	Colas de prioridad (Priority Queue / Heap)	8
2.7	Comparadores en C++ (Comparators)	8
2.8	Vector circular (Circular Buffer)	9
3	Estructuras Hash/Tree	10
3.1	Conjuntos hash (HashSet)	10
3.2	Mapas / Diccionarios hash	11
3.3	Conjuntos ordenados (TreeSet)	11
3.4	Mapas ordenados (TreeMap)	12
4	Trees	12
5	Range Queries	12
5.1	Casos de uso	13
5.2	Prefix Sums	13
5.3	Árbol de Fenwick (BIT)	15
5.4	Sqrt Decomposition	17
5.5	Sparse Table (RMQ)	19
5.6	Segment Tree	20
5.7	RMQ (Range Minimum Query)	22
5.8	Mo's Algorithm	23
5.9	Wavelet Tree	25
5.10	Heavy-Light Decomposition	26
III	Grafos	28
5.11	¿Qué algoritmo usar?	28
6	Traversal	29
6.1	Representación de Grafos	29
6.2	BFS (Breadth-First Search)	30
6.3	BFS en Grafos Desconectados	33
6.4	BFS para Todos los Caminos	33
6.5	DFS	35
6.6	DFS para Conteo de Caminos	36
6.7	DFS + Cerramiento Transitivo	36
6.8	Flood Fill	36
6.9	Laberinto con BFS	36
6.10	Tour de Caballo de Ajedrez	36
6.11	Mínimo Número de Movimientos de Caballo	36
6.12	Serpientes y Escaleras	36

7	Conectividad	36
7.1	Componentes Conexas	36
7.2	Detección de Ciclos	36
7.3	Ciclo en Grafo Dirigido	36
7.4	Puentes	36
7.5	Puntos de Articulación	36
7.6	Componentes Biconectados	36
7.7	Tarjan	36
8	Topological Sort	36
8.1	Kahn's Algorithm	36
8.2	DFS + Orden Topológico	36
8.3	Detección de Ciclos con Topological Sort	36
9	Single Source Shortest Path (SSSP)	36
9.1	BFS para Caminos Más Cortos	36
9.2	0-1 BFS	36
9.3	Dijkstra	36
9.4	Bellman-Ford	36
9.5	Shortest Path en Grafos Binarios	36
9.6	Shortest Path desde Múltiples Orígenes	36
9.7	Shortest Path en Grafos Multietapa	36
9.8	Shortest Path con Exactamente k Aristas	36
10	All Pairs Shortest Path (APSP)	36
10.1	Floyd-Warshall	36
11	Minimum Spanning Tree (MST)	36
11.1	Disjoint Set Union (DSU)	36
11.2	Kruskal	36
11.3	Prim	36
12	Strongly Connected Components (SCC)	36
12.1	Kosaraju	36
12.2	Tarjan SCC	36
12.3	Condensation Graph	36
13	Bipartite Graphs	36
13.1	2-Coloring	36
13.2	Bipartite Matching	36
14	Trees	36
14.1	DFS en Árboles	36
14.2	BFS en Árboles	36
14.3	Diámetro de un Árbol	36
14.4	Centro de un Árbol	36
14.5	Combinatoria en Árboles Ordenados	36
14.6	Tree DP	36
14.7	Rerooting	36
15	Lowest Common Ancestor (LCA)	36
15.1	Binary Lifting	36
15.2	Euler Tour + RMQ	36
15.3	Distancia entre Dos Nodos	36
15.4	k -ésimo Ancestro	36
16	Tree Queries	36
16.1	Euler Tour	37
16.2	Subtree Queries	37
16.3	Path Queries	37
16.4	Heavy-Light Decomposition	37
16.5	Heavy-Light Decomposition + Fenwick	37
16.6	Heavy-Light Decomposition + Segment Tree	37

17 Disjoint Set Union	37
17.1 Union-Find	37
17.2 Path Compression	37
17.3 Union by Size / Rank	37
17.4 Unión de Conjuntos Disjuntos en un Árbol	37
17.5 DSU Offline	37
17.6 DSU Rollback	37
18 Graph Decomposition	37
18.1 Bridge Tree	37
18.2 Block-Cut Tree	37
18.3 Condensation Graph	37
19 Eulerian Graphs	37
19.1 Eulerian Path	37
19.2 Eulerian Circuit	37
19.3 Hierholzer	37
20 Network Flow	37
20.1 Ford-Fulkerson	37
20.2 Edmonds-Karp	37
20.3 Dinic	37
20.4 Min-Cut	37
20.5 Maximum Bipartite Matching	37
21 2-SAT	37
21.1 Implication Graph	37
21.2 2-SAT con SCC	37
22 Backtracking y Problemas Clásicos	37
22.1 Problema de las N Reinas	37
22.2 Variaciones del Problema de las N Reinas	37
22.3 Tour de Caballo	37
23 Advanced Graph Algorithms	37
23.1 Centroid Decomposition	37
23.2 DSU on Tree	37
23.3 Hopcroft-Karp	37
23.4 Min-Cost Max-Flow	37
23.5 DSU Rollback	37
IV Ordenamiento y Búsqueda	37
24 Ordenamiento y Búsqueda (Sorting & Searching)	37
24.1 Número faltante (Missing Number)	37
24.2 Máximo / Mínimo (Array Max/Min)	37
24.3 Par de suma absoluta mínima	37
24.4 Par con diferencia dada (Difference Pair)	37
24.5 Búsqueda ternaria (Ternary Search)	38
24.6 Búsqueda de Fibonacci (Fibonacci Search)	38
24.7 Búsqueda exponencial (Exponential Search)	38
24.8 Búsqueda por saltos (Jump Search)	38
24.9 Heap Sort	38
24.10 QuickSelect	38
25 Búsqueda Binaria (Binary Search)	39
25.1 En arreglo ordenado	39
25.2 Menor x que cumple	39
25.3 Mayor x que cumple	39
25.4 Sobre reales	39
25.5 Con strings	39

V	Matemáticas	39
26	Matemáticas (Math)	39
27	Teoría de Números (Number Theory)	39
VI	Geometría Computacional	39
VII	Programación Dinámica (Dynamic Programming)	39
VIII	Algoritmos Voraces (Greedy)	39
IX	Ad-hoc y Simulación	39
X	String Processing	39
28	Cadenas (Strings)	40
XI	BitWise	40
XII	Game Theory	40

Parte I Introduccion

1 Fundamentos Básicos

1.1 Entrada / Salida Rápida (Fast I/O)

Usar lectura rápida para evitar TLE, hay 3 niveles de lectura rápida, estandarizar Tier 2 y usar Tier 3 (lector por bytes) solo cuando haya del orden de 10^6 – 10^7 lecturas.

Table 1: Casos donde se recomienda Fast I/O

Tipo de problema	Ejemplo
Arrays gigantes	Leer/ordenar/sumar 10^6 – 10^7 enteros
Muchos test cases (T grande)	$T \leq 10^5$, cada uno trivial pero se acumula
Grafos grandes	$M \sim 10^6$ – $2 \cdot 10^6$ aristas para BFS/DFS/Dijkstra/MST
Geometria con muchas consultas	L, S grandes + muchos test cases seguidos hasta EOF
Muchos tokens de texto	Strings/lineas que suman millones de caracteres
Judges clasicos "trampa de I/O"	SPOJ INTEST, ARRAY-SUB, etc.

Regla practica: si N/M es del orden de 10^4 – 10^5 , cin con `sync_with_stdio(false); cin.tie(nullptr);` alcanza de sobra. El FastReader es la "artilleria pesada" para cuando eso deja de ser suficiente.

Tier 1 — Básico: cin / cout.

```
1 int entero; double decimal; string palabra, linea;
2 cin >> entero >> decimal >> palabra; // >> lee UN token
3 // ! cin >> deja el '\n' pendiente;
4 // ! el 1er getline sale VACÍO -> hay que ignorarlo
5 cin.ignore();
6 getline(cin, linea); // línea completa
```

Tier 2 — Rápido: desactiva la sync con C (una vez en main). Punto dulce para casi todo.

```
1 ios_base::sync_with_stdio(false);
2 cin.tie(nullptr); // ! no mezclar con scanf/printf
3 cout << entero << "\n"; // "\n", nunca endl (endl frena)
```

Tier 3 — Más rápido: lector por bytes con fread. Solo con 10^6 – 10^7 lecturas.

```
1 struct FastReader {
2     static const int TAM = 1 << 16; // 64 KB
3     char bufer[TAM];
4     int puntero = 0, leidos = 0;
5
6     int leer() {
7         if (puntero == leidos) {
8             leidos = (int) fread(bufer, 1, TAM, stdin);
9             puntero = 0;
10        }
11        return leidos == 0 ? -1 : bufer[puntero++]; // -1=EOF
12    }
13
14    int nextInt() {
15        int resultado = 0, b = leer();
16        while (b <= ' ') b = leer(); // saltar blancos
17        bool negativo = (b == '-');
18        if (negativo) b = leer();
19        while (b >= '0' && b <= '9') {
20            resultado = resultado * 10 + (b - '0'); b = leer();
21        }
22        return negativo ? -resultado : resultado;
23    }
24 }
```

```
23 long long nextLong() {
24     long long resultado = 0; int b = leer();
25     while (b <= ' ') b = leer();
26     bool negativo = (b == '-');
27     if (negativo) b = leer();
28     while (b >= '0' && b <= '9') {
29         resultado = resultado * 10 + (b - '0'); b = leer();
30     }
31     return negativo ? -resultado : resultado;
32 }
33
34 double nextDouble() {
35     double resultado = 0, divisor = 1; int b = leer();
36     while (b <= ' ') b = leer();
37     bool negativo = (b == '-');
38     if (negativo) b = leer();
39     while (b >= '0' && b <= '9') {
40         resultado = resultado * 10 + (b - '0'); b = leer();
41     }
42     if (b == '.') {
43         b = leer();
44         while (b >= '0' && b <= '9') {
45             resultado += (b - '0') / (divisor *= 10); b = leer();
46         }
47     }
48     return negativo ? -resultado : resultado;
49 }
50
51 string next() { // un token
52     string s; int b = leer();
53     while (b <= ' ') b = leer();
54     while (b > ' ') { s += (char) b; b = leer(); }
55     return s;
56 }
57
58 string nextLine() { // línea completa
59     string s; int b = leer();
60     while (b != '\n' && b != -1) {
61         if (b != '\r') s += (char) b; b = leer();
62     }
63     return s;
64 }
65 }
```

1.2 Formateo de Salida

La función `printf` utiliza una cadena de formato para indicar cómo debe interpretarse y mostrarse cada argumento. Cada especificador comienza con `%` y corresponde al argumento que aparece después de la cadena.

```
1 // %d -> entero (int)
2 printf("%d\n", 42);
3 // Entrada: 42
4 // Salida: 42
5
6 // %lld -> entero largo (long long)
7 long long x = 1234567890123LL;
8 printf("%lld\n", x);
9 // Entrada: 1234567890123
10 // Salida: 1234567890123
11
12 // %5d -> ancho mínimo de 5 caracteres, alineado a la derecha
13 printf("%5d\n", 42);
14 // Entrada: 42
15 // Salida: "   42"
16
17 // %-5d -> ancho mínimo de 5 caracteres, alineado a la izquierda
18 printf("%-5d\n", 42);
19 // Entrada: 42
20 // Salida: "42   "
21
22 // %05d -> ancho 5, completando con ceros
23 printf("%05d\n", 42);
24 // Entrada: 42
25 // Salida: "00042"
26
27 // %+d -> muestra siempre el signo
28 printf("%+d\n", 42);
29 // Entrada: 42
30 // Salida: "+42"
31
32 // %x -> hexadecimal usando letras minúsculas
33 printf("%x\n", 255);
34 // Entrada: 255
35 // Salida: ff
36
37 // %X -> hexadecimal usando letras mayúsculas
38 printf("%X\n", 255);
39 // Entrada: 255
40 // Salida: FF
41 }
```

```

42 // %.2f -> numero real con exactamente 2 decimales
43 printf("%.2f\n", 3.14159);
44 // Entrada: 3.14159
45 // Salida: 3.14
46
47 // %.0f -> numero real sin decimales (redondea)
48 printf("%.0f\n", 3.7);
49 // Entrada: 3.7
50 // Salida: 4
51
52 // %e -> notacion cientifica
53 printf("%e\n", 31415.9);
54 // Entrada: 31415.9
55 // Salida: 3.141590e+04
56
57 // %c -> un solo caracter
58 printf("%c\n", 'A');
59 // Entrada: 'A'
60 // Salida: A
61
62 // %s -> cadena de caracteres
63 printf("%s\n", "hola");
64 // Entrada: "hola"
65 // Salida: hola
66
67 // Se pueden combinar varios especificadores
68 printf("%d %lld %.2f\n", 42, 10000000000LL, 3.14159);
69 // Entrada: 42, 10000000000, 3.14159
70 // Salida: 42 10000000000 3.14
71
72 // %% -> imprime un '%' literal
73 printf("100%%\n");
74 // Entrada: no recibe ningun argumento para %
75 // Salida: 100%
76
77 // ! NO uses %n en C/C++ (puede ser peligroso)
78 // Para scanf, un double se lee con %lf

```

Estructura de un especificador

Un formato puede combinar varias partes:

```

1 // % [flags] [width] [.precision] [specifier]
2 // % 0 5 .2 f
3
4 printf("%05d\n", 42);
5 // % -> comienza el formato
6 // 0 -> rellena con ceros
7 // 5 -> ancho minimo de 5 caracteres
8 // d -> interpreta el argumento como int
9 //
10 // Salida: 00042

```

Los modificadores más utilizados en programación competitiva son:

Formato	Tipo	Ejemplo de salida
%d	int	42
%lld	long long	1234567890123
%c	char	A
%s	string/char*	hola
%f	double	3.140000
%.2f	double	3.14
%x	int	ff
%X	int	FF

Regla práctica: en programación competitiva, los formatos más importantes son %d para int, %lld para long long, %c para caracteres, %s para cadenas y %.2f (o una precisión diferente) para números reales.

1.3 Condicionales y Ciclos

```

1 // if / else: ejecuta un bloque segun una condicion
2 if (x > 0) cout << "pos"; // x > 0 -> "pos"
3 else if (x < 0) cout << "neg"; // x < 0 -> "neg"
4 else cout << "cero"; // x == 0 -> "cero"
5
6 // Operador ternario: condicion ? valor_si_true : valor_si_false
7 string s = (x >= 0) ? "no neg" : "neg";
8 // Si x >= 0, s = "no neg"; si no, s = "neg"

```

```

9
10 // switch: compara una expresion con diferentes valores
11 // Solo funciona directamente con tipos integrales como int o
   char,
12 // NO con string.
13 switch (x) {
14     case 1: /*...*/ break; // x == 1
15     case 2: case 3: /*...*/ break; // x == 2 o x == 3
16     default: /*...*/; // ningun case coincide
17 }
18 // ! Sin break, se continua ejecutando el siguiente case (fall-through)
19
20 // C++17: se puede declarar una variable dentro del if
21 if (int r = x % 2; r == 0) {}
22 // r solo existe dentro del if y sus bloques asociados
23
24 // for clasico: usa un indice y se repite mientras i < n
25 for (int i = 0; i < n; i++) {}
26
27 // range-based for: recorre directamente los elementos
28 // No proporciona el indice.
29 for (int v : datos) {}
30
31 // & obtiene una referencia al elemento original.
32 // Por tanto, las modificaciones afectan al vector.
33 for (int &v : datos) v *= 2;
34
35 // while: ejecuta mientras la condicion sea verdadera
36 while (c < n) c++;
37
38 // do-while: primero ejecuta el bloque y DESPUES verifica la
   condicion.
39 // Por eso se ejecuta AL MENOS una vez.
40 do {
41     c--;
42 } while (c > 0);
43
44 // break -> termina inmediatamente el ciclo o switch
45 // continue -> salta a la siguiente iteracion del ciclo
46
47 // ! C++ NO tiene break con etiqueta.
48 // Para salir de ciclos anidados se puede usar una bandera,
49 // encapsular la logica en una funcion y usar return, etc.

```

1.4 Conversiones, Parseos y Casteos

```

1 // string -> numero (parseo)
2 // Convierte una cadena de texto en el tipo numerico indicado.
3 int e = stoi("42");
4 // Entrada: "42" -> Salida: 42
5
6 long long l = stoll("10000000000");
7 // Entrada: "10000000000" -> Salida: 10000000000
8
9 double d = stod("3.14");
10 // Entrada: "3.14" -> Salida: 3.14
11
12 // Tambien se puede indicar la base numerica.
13 int bin = stoi("1010", nullptr, 2);
14 // "1010" en base 2 -> 10
15
16 int hex = stoi("ff", nullptr, 16);
17 // "ff" en base 16 -> 255
18
19
20 // numero -> string
21 // Convierte cualquier numero compatible en texto.
22 string s = to_string(42);
23 // Entrada: 42 -> Salida: "42"
24
25
26 // Casteo: fuerza la conversion de un tipo a otro.
27 // Al pasar de decimal a entero, se TRUNCa hacia 0.
28 int t = (int) 3.99;
29 // 3.99 -> 3
30
31 int r = static_cast<int>(3.99);
32 // 3.99 -> 3
33 // static_cast<int> es la forma recomendada en C++
34
35
36 // ! int / int = division ENTERA
37 double mal = 7 / 2;
38 // 7 y 2 son int -> 3
39 // Luego 3 se convierte a double -> 3.0
40
41 double bien = 7 / 2.0;
42 // 2.0 es double -> division decimal
43 // 7 / 2.0 -> 3.5
44

```

```

45 // caracteres
46 // Los char tienen un valor numerico asociado (ASCII).
47 int cod = (int) 'A';
48 // 'A' -> 65
49
50
51 char ch = (char) 66;
52 // 66 -> 'B'
53
54 // Truco muy usado para convertir un digito char a int.
55 int dig = '7' - '0';
56 // '7' tiene valor 55 y '0' tiene valor 48
57 // 55 - 48 -> 7

```

Parte II Estructuras de Datos

2 Basic Structures

```

1 int a[5] = {1,2,3,4,5}; // crear con valores: O(n)
2
3 int b[5];                // crear de tamaño 5
4                          // posiciones: b[0] ... b[4]
5                          // ! Valores no inicializados
6
7 b[0] = 10;               // INSERT: O(1)
8 b[1] = 20;               // INSERT: O(1)
9
10 int x = b[0];            // READ: O(1)
11
12 b[0] = 30;               // UPDATE: O(1)
13
14 for (int i = 0; i < 5; i++)
15     cout << b[i] << " "; // READ: recorrer -> O(n)
16
17 // DELETE: no existe metodo para eliminar
18 // Se puede sobrescribir o manejar logicamente
19 b[0] = 0;                // DELETE logico: O(1)
20
21 // ! El tamaño de un array estatico no cambia.
22 // Para mas posiciones se necesita otro arreglo.

```

2.1 Matrices (Arreglos 2D)

Arreglo bidimensional (filas $\times$ columnas). **Complejidad:** acceso $[i][j]$ $O(1)$; crear $n \times m$ en $O(nm)$.

```

1 vector<vector<int>> m(3, vector<int>(4,0)); // CREATE: O(n*m)
2 // matriz 3x4, todas las posiciones inicializadas en 0
3
4 m[1][2] = 7;             // INSERT: O(1)
5
6 int x = m[1][2];         // READ: O(1)
7
8 m[1][2] = 10;            // UPDATE: O(1)
9
10 for (int i = 0; i < 3; i++)
11     for (int j = 0; j < 4; j++)
12         cout << m[i][j] << " ";
13 // READ: recorrer toda la matriz -> O(n*m)
14
15 // DELETE: no existe metodo para eliminar una posicion.
16 // Se puede sobrescribir el valor o manejarlo logicamente.
17 m[1][2] = 0;             // DELETE logico: O(1)
18
19 // ! El tamaño de la matriz puede cambiar con vector,
20 // pero un arreglo 2D estatico no puede cambiar de tamaño.

```

2.2 Vectores / Listas dinámicas

Arreglo que **crece** solo (ArrayList / vector / list). **Complejidad:** acceso por índice $O(1)$; agregar al final $O(1)$ amortizado; insertar/borrar en medio $O(n)$; buscar $O(n)$.

```

1 vector<int> v;            // CREATE: lista vacia
2
3 v.push_back(3);          // INSERT: final -> O(1) amortizado
4 v.push_back(7);          // INSERT: final -> O(1) amortizado
5
6 int x = v[0];            // READ: acceso -> O(1)

```

```

7 int n = v.size();        // READ: tamaño -> O(1)
8
9 v[0] = 10;               // UPDATE: modificar -> O(1)
10
11 v.insert(v.begin() + 1, 5); // INSERT: medio -> O(n)
12
13 v.erase(v.begin() + 1);  // DELETE: medio -> O(n)
14
15 // Buscar un elemento recorriendo el vector
16 for (int x : v)
17     if (x == 10) { /* ... */ }
18 // SEARCH: recorrer -> O(n)
19
20 // DELETE: eliminar el ultimo elemento
21 v.pop_back();            // DELETE: final -> O(1)
22
23 // ! vector mantiene acceso por índice O(1).
24 // ! Al crecer puede realocar su memoria internamente.

```

2.3 Listas enlazadas (LinkedList)

Nodos enlazados. **Complejidad:** insertar/eliminar en los **extremos** $O(1)$; acceso por índice $O(n)$; buscar $O(n)$. Útil como cola o deque.

```

1 list<int> l;             // CREATE: lista vacia
2
3 l.push_front(1);         // INSERT: inicio -> O(1)
4 l.push_back(9);          // INSERT: final -> O(1)
5
6 int x = l.front();        // READ: primer elemento -> O(1)
7 int y = l.back();         // READ: ultimo elemento -> O(1)
8
9 l.front() = 5;           // UPDATE: primer elemento -> O(1)
10 l.back() = 10;           // UPDATE: ultimo elemento -> O(1)
11
12 l.pop_front();           // DELETE: inicio -> O(1)
13 l.pop_back();            // DELETE: final -> O(1)
14
15 // ! No existe acceso directo por índice.
16 // Avanzar hasta una posicion requiere recorrer nodos.
17 auto it = l.begin();
18 advance(it, 2);          // acceso al tercer elemento -> O(n)
19
20 // Buscar un elemento requiere recorrer la lista.
21 for (int x : l)
22     if (x == 10) { /* ... */ }
23 // SEARCH: recorrer -> O(n)
24
25 // ! list no permite l[i].
26 // Para acceso por índice, vector suele ser mejor.

```

2.4 Pilas (Stack)

Estructura **LIFO** (último en entrar, primero en salir). **Complejidad:** insertar, eliminar y consultar el tope $O(1)$. Para DFS iterativo, paréntesis balanceados y deshacer.

```

1 stack<int> pila;         // CREATE: pila vacia
2
3 pila.push(3);            // INSERT: O(1)
4 pila.push(7);            // INSERT: O(1)
5
6 int tope = pila.top();    // READ: consultar tope -> O(1)
7
8 pila.top() = 10;         // UPDATE: modificar tope -> O(1)
9
10 pila.pop();              // DELETE: eliminar tope -> O(1)
11
12 // ! Solo se puede acceder directamente al elemento superior.
13 // ! No existe acceso por índice.

```

2.5 Colas (Queue)

Estructura **FIFO** (primero en entrar, primero en salir). **Complejidad:** encolar y desencolar $O(1)$. Base del BFS.

```

1 queue<int> cola;         // CREATE: cola vacia
2
3 cola.push(3);            // INSERT: final -> O(1)

```



```

4 cola.push(7);           // INSERT: final -> O(1)
5
6 int frente = cola.front(); // READ: primero -> O(1)
7 int atrás = cola.back();  // READ: último -> O(1)
8
9 cola.front() = 10;       // UPDATE: modificar primero -> O(1)
10
11 cola.pop();             // DELETE: primero -> O(1)
12
13 // ! push agrega al final y pop elimina del frente.
14 // ! No existe acceso directo por índice.

```

2.6 Colas de prioridad (Priority Queue / Heap)

Devuelve siempre el elemento de mayor (o menor) prioridad. **Complejidad:** insertar y extraer $O(\log n)$; consultar el tope $O(1)$. Para Dijkstra, Prim, k-ésimo mayor.

```

1 priority_queue<int> pq; // CREATE: MAX-heap por defecto
2
3 pq.push(3);           // INSERT: O(log n)
4 pq.push(7);           // INSERT: O(log n)
5
6 int mx = pq.top();     // READ: mayor -> O(1)
7
8 pq.pop();             // DELETE: mayor -> O(log n)
9
10 // min-heap: el menor queda en el tope
11 priority_queue<int, vector<int>, greater<int>> pqMin;
12
13 pqMin.push(3);         // INSERT: O(log n)
14 int mn = pqMin.top();  // READ: menor -> O(1)
15 pqMin.pop();           // DELETE: O(log n)
16
17 // ! No existe acceso directo a elementos intermedios.
18 // ! priority_queue no tiene una operacion UPDATE directa.

```

2.7 Comparadores en C++ (Comparators)

Idea: un comparador es una función (lambda, functor, o `operator<` sobrecargado) que le dice a `sort`, `set`, `map`, `priority_queue`, etc. **cómo ordenar** dos elementos. Debe cumplir la propiedad de *orden estricto débil* (*strict weak ordering*): devolver `true` si el primer argumento va **estrictamente antes** que el segundo, y `false` en cualquier otro caso —incluyendo cuando se consideran “iguales” para efectos de orden. El error más común es usar `<=` en vez de `<`: eso rompe la propiedad y produce comportamiento indefinido (desde resultados mal ordenados hasta crashes).

¿Por qué usarlo? El orden por defecto casi nunca alcanza: necesitas ordenar por varios criterios a la vez, en orden descendente, por una clave derivada (no el valor mismo), o con un `struct`/`pair` propio que no debe usar el orden lexicográfico por defecto.

Casos de uso:

- Ordenar por múltiples campos (ej. por costo, y en empate por índice original).
- `priority_queue` como min-heap en vez de max-heap (o con un orden custom sobre un `struct`).
- `set`/`map` con orden propio sobre un tipo definido por ti.
- Ordenar **índices** según los valores de otro arreglo, sin mover el arreglo original.

Complejidad: el comparador en sí no cambia la complejidad del algoritmo que lo usa (`sort` sigue siendo $O(n \log n)$); pero un comparador que no sea un orden estricto débil puede producir UB o bucles infinitos.

Comparador con lambda para sort (multi-criterio, índices)

```

1 struct Persona {
2     string nombre;
3     int edad;
4 };
5
6 vector<Persona> personas = {
7     {"Ana", 30}, {"Luis", 25}, {"Eva", 25}
8 };
9
10 // ordena por edad ascendente; en empate, por nombre alfabético
11 sort(personas.begin(), personas.end(),
12     [](const Persona& a, const Persona& b) {
13         if (a.edad != b.edad)
14             return a.edad < b.edad;
15         return a.nombre < b.nombre;
16     });
17 // resultado: Eva(25), Luis(25), Ana(30)
18
19 // ordenar en orden DESCENDENTE: solo invierte la comparación
20 sort(personas.begin(), personas.end(),
21     [](const Persona& a, const Persona& b) {
22         return a.edad > b.edad;
23     });
24 // resultado: Ana(30), luego Luis/Eva(25) en su orden relativo
    previo

```

Ordenar índices en vez del arreglo (muy común):

```

1 vector<int> valores = {40, 10, 30, 20};
2
3 vector<int> idx(valores.size());
4 iota(idx.begin(), idx.end(), 0); // idx = [0, 1, 2, 3]
5
6 sort(idx.begin(), idx.end(),
7     [&](int a, int b) { return valores[a] < valores[b]; });
8 // idx queda [1, 3, 2, 0]
9 // (valores[1]=10, valores[3]=20, valores[2]=30, valores[0]=40)
10
11 for (int i : idx)
12     cout << valores[i] << " ";
13 // imprime: 10 20 30 40
14 // sin haber movido "valores" del lugar

```

operator< dentro de un struct (orden por defecto)

Idea: si sobrecargas `operator<` dentro de tu propio tipo, ese es el orden que usan automáticamente `sort`, `set`, `map` (como clave) y `priority_queue` **sin pasar ningún comparador extra**.

```

1 struct Punto {
2     int x, y;
3
4     bool operator<(const Punto& otro) const {
5         if (x != otro.x) return x < otro.x;
6         return y < otro.y;
7     }
8 };
9
10 set<Punto> s;
11 s.insert({3, 4});
12 s.insert({1, 2});
13 s.insert({1, 5});
14
15 for (auto& p : s)
16     cout << "(" << p.x << ", " << p.y << ") ";
17 // imprime en orden: (1,2) (1,5) (3,4)
18 // -> set los ordeno automáticamente usando operator<

```

priority_queue con comparador custom

Idea: `priority_queue` es un **max-heap** por defecto (el elemento “mayor” según el comparador queda arriba). Para un min-heap con tipos simples, se usa `greater<T>`. Para un `struct` propio con una regla de prioridad distinta a `operator<`, se pasa un **functor** (struct con `operator()`).

```

1 // min-heap de enteros: el menor queda arriba
2 priority_queue<int, vector<int>, greater<int>> minHeap;
3 minHeap.push(5);
4 minHeap.push(1);
5 minHeap.push(3);
6
7 cout << minHeap.top();
8 // 1 (el menor, no el mayor)

```

```

1 struct Persona {
2     string nombre;
3     int edad;
4 };
5
6 // functor: queda arriba la persona con MENOR edad
7 // (priority_queue es max-heap, así que se invierte el sentido:
8 // operator() debe devolver true si "a" tiene MENOR prioridad que
9 // "b")
10 struct CompararPorEdad {
11     bool operator()(const Persona& a, const Persona& b) const {
12         return a.edad > b.edad;
13     }
14 };
15 priority_queue<Persona, vector<Persona>, CompararPorEdad> pq;
16 pq.push({"Ana", 30});
17 pq.push({"Luis", 20});
18 pq.push({"Eva", 25});
19
20 cout << pq.top().nombre;
21 // "Luis" (la persona mas joven queda en la cima)

```

Reglas de orden estricto débil (*strict weak ordering*)

Todo comparador `cmp(a, b)` debe cumplir:

- **Irreflexividad:** `cmp(a, a)` siempre `false`. Un elemento nunca va estrictamente antes que sí mismo.
- **Asimetría:** si `cmp(a, b)` es `true`, entonces `cmp(b, a)` debe ser `false`.
- **Transitividad:** si `cmp(a, b)` y `cmp(b, c)` son `true`, entonces `cmp(a, c)` también debe serlo.
- **Consistencia en los empates:** si ni `cmp(a, b)` ni `cmp(b, a)` son `true`, se consideran “equivalentes” para efectos de orden (aunque no sean idénticos como objetos).

El error más común:

```

1 // INCORRECTO: usa <= en vez de <
2 sort(v.begin(), v.end(), [](int a, int b) {
3     return a <= b; // viola irreflexividad: cmp(a,a) da true
4 });
5 // esto es Undefined Behavior: en el mejor caso ordena mal,
6 // en el peor caso crashea o hace un bucle infinito.
7
8 // CORRECTO:
9 sort(v.begin(), v.end(), [](int a, int b) {
10     return a < b;
11 });

```

Otro error común: comparadores que no son transitivos al combinar varios criterios sin desempate consistente (ej. comparar solo por `a.x < b.x` cuando a veces `a.x == b.x` pero querías tratar esos casos con otro criterio). Si dos elementos “empatan” en el primer criterio, siempre hay que **decidir** con un segundo criterio, o dejarlos expresamente como equivalentes.

Regla práctica: si tu comparador tiene la forma `return a.campo1 < b.campo1;` y sospechas que puede haber empates que te importan, agrega el siguiente criterio explícitamente (`if (a.campo1 != b.campo1) return ...; return a.campo2 < b.campo2;`) en vez

de dejar el desempate al azar del algoritmo de ordenamiento (que no es estable a menos que uses `stable_sort`).

2.8 Vector circular (Circular Buffer)

Idea: un arreglo de tamaño fijo (**cap**) que se comporta como una **cola doble**. En lugar de mover elementos cuando se inserta o elimina, se reutilizan las posiciones del arreglo de forma circular.

Se mantienen tres variables principales:

- **cap:** capacidad total del arreglo.
- **ini:** posición física donde comienza el primer elemento.
- **tam:** cantidad actual de elementos almacenados.

La posición lógica i no coincide necesariamente con la posición física del arreglo. Se obtiene mediante:

$$\text{posicionFisica}(i) = (\text{ini} + i) \% \text{cap}$$

Por ejemplo, con `cap = 5` y `ini = 3`:

Índice lógico	0	1	2	3	4
Índice físico	3	4	0	1	2

Cuando se llega al final del arreglo, el módulo hace que la siguiente posición vuelva al comienzo. Por eso **no es necesario mover los elementos**.

¿Por qué usarlo? En un arreglo normal, insertar al frente requiere desplazar todos los elementos y cuesta $O(n)$. En un vector circular solo se mueve `ini`, por lo que cuesta $O(1)$.

Casos de uso:

- Colas de tamaño fijo.
- Buffers de audio, video o red.
- Ventanas deslizantes.
- Sistemas donde llegan y salen datos continuamente.
- Simulaciones de procesos en una cola.
- Mantener únicamente los últimos k elementos.

Si el buffer está lleno y se ejecuta `pushBack`, el nuevo elemento **reemplaza al más antiguo**. Esto permite mantener automáticamente una ventana de tamaño fijo.

Complejidad: insertar, eliminar, consultar extremos, acceder por posición y comprobar el tamaño son $O(1)$.

```

1 struct VectorCircular {
2     vector<int> buf;
3     int cap, ini = 0, tam = 0;
4
5     VectorCircular(int c) : buf(c), cap(c) {}
6
7     // CREATE: crea un buffer con capacidad fija.
8     // Los elementos inicialmente no están ocupados.
9
10    bool empty() {
11        return tam == 0;
12    } // O(1)
13
14    bool full() {
15        return tam == cap;
16    } // O(1)
17

```

```

18 int size() {
19     return tam;
20 } // O(1)
21
22 int capacity() {
23     return cap;
24 } // O(1)
25
26 // Convierte una posicion logica en fisica.
27 int pos(int i) {
28     return (ini + i) % cap;
29 } // O(1)
30
31 // INSERT: agrega al final.
32 void pushBack(int x) {
33     buf[(ini + tam) % cap] = x;
34
35     if (tam < cap)
36         tam++;
37     else
38         ini = (ini + 1) % cap;
39 } // O(1)
40
41 // INSERT: agrega al inicio.
42 void pushFront(int x) {
43     ini = (ini - 1 + cap) % cap;
44     buf[ini] = x;
45
46     if (tam < cap)
47         tam++;
48 } // O(1)
49
50 // READ: consulta el primer elemento.
51 int front() {
52     return buf[ini];
53 } // O(1)
54
55 // READ: consulta el ultimo elemento.
56 int back() {
57     return buf[(ini + tam - 1) % cap];
58 } // O(1)
59
60 // READ: acceso por posicion logica.
61 int at(int i) {
62     return buf[(ini + i) % cap];
63 } // O(1)
64
65 // UPDATE: modifica un elemento.
66 void set(int i, int x) {
67     buf[(ini + i) % cap] = x;
68 } // O(1)
69
70 // DELETE: elimina el primer elemento.
71 int popFront() {
72     int v = buf[ini];
73     ini = (ini + 1) % cap;
74     tam--;
75     return v;
76 } // O(1)
77
78 // DELETE: elimina el ultimo elemento.
79 int popBack() {
80     tam--;
81     return buf[(ini + tam) % cap];
82 } // O(1)
83
84 // DELETE: elimina todos los elementos.
85 void clear() {
86     ini = 0;
87     tam = 0;
88 } // O(1)
89
90 // SEARCH: busca un elemento.
91 // A diferencia de las operaciones anteriores,
92 // hay que recorrer los elementos.
93 bool contains(int x) {
94     for (int i = 0; i < tam; i++)
95         if (at(i) == x)
96             return true;
97
98     return false;
99 } // O(n)
100
101 // READ: recorrer los elementos en orden logico.
102 void print() {
103     for (int i = 0; i < tam; i++)
104         cout << at(i) << " ";
105 } // O(n)
106 };

```

Ejemplo de funcionamiento:

```
1 VectorCircular v(5);
```

```

2 v.pushBack(10);
3 v.pushBack(20);
4 v.pushBack(30);
5 // [10, 20, 30, _, _]
6 // ini = 0, tam = 3
7
8 v.pushFront(5);
9 // [5, 10, 20, 30, _]
10 // ini cambia de 0 a 4
11
12 cout << v.front();
13 // 5
14
15 cout << v.back();
16 // 30
17
18 v.popFront();
19 // elimina 5
20 // [_, 10, 20, 30, _]
21
22 v.pushBack(40);
23 v.pushBack(50);
24 // [_, 10, 20, 30, 40, 50]
25 // El almacenamiento puede haber dado la vuelta.
26
27 v.pushBack(60);
28 // El buffer estaba lleno.
29 // Se agrega 60 y se elimina el mas antiguo.
30 // Resultado logico: [20, 30, 40, 50, 60]
31

```

Ventaja principal: el arreglo físico puede verse “desordenado”, pero los elementos siempre tienen un orden lógico determinado por `ini`. Por ejemplo:

```

1 // Memoria fisica:
2 // [40, 50, 20, 30, _]
3 //      ^
4 //      ini = 2
5
6 // Orden logico:
7 // 20 -> 30 -> 40 -> 50

```

No importa dónde estén físicamente los elementos: `at(0)` siempre representa el primero, `at(1)` el segundo, y así sucesivamente.

Diferencia con un vector normal:

Operación	vector	Circular
Agregar al final	$O(1)^*$	$O(1)$
Eliminar al final	$O(1)$	$O(1)$
Agregar al frente	$O(n)$	$O(1)$
Eliminar al frente	$O(n)$	$O(1)$
Acceso por índice	$O(1)$	$O(1)$
Buscar	$O(n)$	$O(n)$

*Amortizado.

Regla práctica: si necesitas una estructura de tamaño fijo donde los elementos entran y salen continuamente por los extremos, un vector circular evita los desplazamientos de un arreglo normal y mantiene las operaciones principales en $O(1)$.

3 Estructuras Hash/Tree

3.1 Conjuntos hash (HashSet)

Colección de elementos **sin duplicados** y sin orden garantizado. En C++ se implementa mediante `unordered_set`. Utiliza una tabla hash, por lo que las operaciones son $O(1)$ en promedio. Es útil cuando solo interesa saber si un elemento existe, sin necesidad de mantener los elementos ordenados.

Complejidad: insertar, buscar y borrar $O(1)$ promedio ($O(n)$ peor caso).

```

1 unordered_set<int> s;           // CREATE: conjunto vacio
2
3 s.insert(3);                   // INSERT: O(1) prom.
4 s.insert(7);                   // INSERT: O(1) prom.
5 s.insert(3);                   // no se agrega: ya existe
6
7 bool hay = s.count(3);         // SEARCH: O(1) prom.
8 // hay = 1 si existe, 0 si no existe
9
10 auto it = s.find(7);           // SEARCH: O(1) prom.
11 // devuelve iterador al elemento o s.end()
12
13 s.erase(3);                   // DELETE: O(1) prom.
14
15 int n = s.size();              // READ: cantidad -> O(1)
16 bool vacio = s.empty();        // READ: ¿esta vacio? -> O(1)
17
18 // Recorrer todos los elementos
19 for (int x : s)
20     cout << x << " ";
21 // O(n), pero el orden NO esta garantizado.
22
23 // ! No existe acceso por indice: s[0] no es valido.
24 // ! Los elementos repetidos se ignoran.
25 // ! El orden puede cambiar entre ejecuciones.

```

Cuándo usarlo:

- Saber rápidamente si un elemento ya apareció.
- Eliminar duplicados.
- Mantener un conjunto de elementos visitados.
- Detectar elementos repetidos en $O(n)$ promedio.
- BFS/DFS: guardar vértices visitados cuando sea conveniente.

3.2 Mapas / Diccionarios hash

Estructura que almacena pares **clave** $\rightarrow$ **valor**. En C++ se implementa mediante `unordered_map`. Cada clave identifica un valor y **no puede repetirse**. Si se asigna un nuevo valor a una clave existente, se reemplaza el anterior.

Complejidad: insertar, acceder, buscar y borrar por clave $O(1)$ promedio ($O(n)$ peor caso).

```

1 unordered_map<string,int> m;    // CREATE: mapa vacio
2
3 m["a"] = 1;                    // INSERT: O(1) prom.
4 m["b"] = 2;                    // INSERT: O(1) prom.
5
6 int v = m["a"];                // READ: acceder -> O(1) prom.
7 // v = 1
8
9 m["a"] = 10;                   // UPDATE: O(1) prom.
10 // la clave "a" ahora tiene valor 10
11
12 m.erase("a");                 // DELETE: O(1) prom.
13
14 bool existe = m.count("b");    // SEARCH: O(1) prom.
15 // existe = 1
16
17 auto it = m.find("b");         // SEARCH: O(1) prom.
18 if (it != m.end())
19     cout << it->second;        // valor asociado a "b"
20
21 // Recorrer todas las parejas
22 for (auto [clave, valor] : m)
23     cout << clave << " " << valor;
24 // O(n), sin orden garantizado.

```

Importante: usar `m[clave]` para consultar una clave que no existe puede **crear esa clave** con su valor por defecto. Para comprobar existencia sin insertar, es preferible usar `count()` o `find()`.

```

1 unordered_map<string,int> m;
2
3 int x = m["a"];
4 // Si "a" no existia, se crea:

```

```

5 // "a" -> 0
6 // ! operator[] puede INSERTAR una clave al consultar.
7
8 // count(): comprueba si existe SIN insertar
9 bool existe = m.count("b");
10 // Devuelve 0 si no existe, 1 si existe.
11
12 // find(): busca la clave SIN insertar
13 auto it = m.find("b");
14
15 if (it != m.end())
16     cout << it->second;        // valor asociado a "b"
17 else
18     cout << "No existe";
19
20 // ! count() solo indica si existe.
21 // ! find() permite acceder a la pareja clave -> valor.

```

Uso clásico: contar frecuencias.

```

1 unordered_map<int,int> freq;
2
3 for (int x : datos)
4     freq[x]++;                // frecuencia de cada elemento
5
6 cout << freq[5];
7 // cantidad de veces que aparece 5

```

Cuándo usarlo:

- Contar frecuencias.
- Asociar identificadores con información.
- Buscar rápidamente un valor a partir de una clave.
- Problemas de conteo y frecuencia.
- Memorizar resultados de estados o funciones.

3.3 Conjuntos ordenados (TreeSet)

Conjunto de elementos **sin duplicados y ordenados**. En C++ se implementa mediante `set`, normalmente como un árbol balanceado.

A diferencia de `unordered_set`, los elementos siempre se mantienen ordenados según el comparador utilizado.

Complejidad: insertar, buscar y borrar $O(\log n)$. Consultar el menor o mayor elemento mediante `begin()` o `rbegin()` es $O(1)$.

```

1 set<int> s;                     // CREATE: conjunto vacio
2
3 s.insert(5);                   // INSERT: O(log n)
4 s.insert(1);                   // INSERT: O(log n)
5 s.insert(3);                   // INSERT: O(log n)
6 s.insert(3);                   // no se agrega: ya existe
7
8 int menor = *s.begin();        // READ: minimo -> O(1)
9 int mayor = *s.rbegin();       // READ: maximo -> O(1)
10
11 auto it = s.find(3);           // SEARCH: O(log n)
12 if (it != s.end())
13     cout << *it;
14
15 s.erase(3);                   // DELETE: O(log n)
16
17 int n = s.size();              // READ: cantidad -> O(1)
18
19 // Recorrer en orden ascendente
20 for (int x : s)
21     cout << x << " ";
22 // O(n)

```

lower_bound y upper_bound:

```

1 set<int> s = {1, 3, 5, 7, 9};
2
3 auto it1 = s.lower_bound(5);
4 // primer elemento >= 5
5 // resultado: 5
6
7 auto it2 = s.upper_bound(5);
8 // primer elemento > 5
9 // resultado: 7

```

Esto permite resolver consultas de **piso y techo** rápidamente:

```
1 // FLOOR: mayor elemento <= x
2 auto it = s.upper_bound(x);
3
4 if (it != s.begin()) {
5     --it;
6     cout << *it;
7 }
8
9 // CEILING: menor elemento >= x
10 auto it2 = s.lower_bound(x);
11
12 if (it2 != s.end())
13     cout << *it2;
```

Cuándo usarlo:

- Cuando se necesitan elementos únicos y ordenados.
- Obtener mínimo o máximo dinámicamente.
- Encontrar el siguiente elemento mayor o igual.
- Encontrar el anterior elemento menor o igual.
- Mantener un conjunto mientras se insertan y eliminan elementos.

Diferencia clave: si solo necesitas saber si existe un elemento, `unordered_set` suele ser más rápido. Si necesitas consultar orden, `lower_bound`, `upper_bound`, mínimo o máximo, `set` es la opción adecuada.

3.4 Mapas ordenados (TreeMap)

Mapa **clave** → **valor** cuyas claves se mantienen ordenadas. En C++ se implementa mediante `map`, usando un árbol balanceado.

Cada clave aparece una sola vez. Los valores pueden repetirse.

Complejidad: insertar, buscar, acceder y borrar por clave $O(\log n)$. `begin()` permite obtener la menor clave en $O(1)$.

```
1 map<string,int> m; // CREATE: mapa vacío
2
3 m["b"] = 2; // INSERT: O(log n)
4 m["a"] = 1; // INSERT: O(log n)
5 m["c"] = 3; // INSERT: O(log n)
6
7 int v = m["a"]; // READ: O(log n)
8 // v = 1
9
10 m["a"] = 10; // UPDATE: O(log n)
11
12 m.erase("b"); // DELETE: O(log n)
13
14 auto it = m.find("c"); // SEARCH: O(log n)
15 if (it != m.end())
16     cout << it->second;
17
18 // Primera pareja: menor clave
19 auto primera = m.begin(); // O(1)
20 cout << primera->first;
21
22 // Última pareja: mayor clave
23 auto ultima = prev(m.end()); // O(1) amortizado
24 cout << ultima->first;
```

Las claves se recorren ordenadas:

```
1 map<int,string> m;
2
3 m[30] = "C";
4 m[10] = "A";
5 m[20] = "B";
6
7 for (auto [clave, valor] : m)
8     cout << clave << " " << valor;
```

```
10 // Salida:
11 // 10 A
12 // 20 B
13 // 30 C
```

`lower_bound` y `upper_bound` también funcionan sobre las claves:

```
1 map<int,string> m;
2 m[10] = "A";
3 m[20] = "B";
4 m[30] = "C";
5
6 auto it = m.lower_bound(15);
7 // primera clave >= 15
8 // apunta a 20
9
10 auto it2 = m.upper_bound(20);
11 // primera clave > 20
12 // apunta a 30
```

Uso clásico: contar frecuencias ordenadas.

```
1 map<int,int> freq;
2
3 for (int x : datos)
4     freq[x]++;
5
6 // Las claves aparecen en orden creciente.
7 for (auto [x, cantidad] : freq)
8     cout << x << ": " << cantidad << "\n";
```

Cuándo usarlo:

- Cuando se necesitan pares clave-valor ordenados.
- Contar frecuencias y procesarlas en orden.
- Encontrar claves inmediatamente mayores o menores.
- Procesar eventos o valores manteniendo un orden dinámico.
- Cuando se necesitan operaciones de árbol como `lower_bound` y `upper_bound`.

Comparación rápida

Estructura	Duplicados	Orden	Buscar
<code>unordered_set</code>	No	No	$O(1)$ prom.
<code>unordered_map</code>	Claves: No	No	$O(1)$ prom.
<code>set</code>	No	Sí	$O(\log n)$
<code>map</code>	Claves: No	Sí	$O(\log n)$

Regla práctica:

- `unordered_set` → ¿Ya existe este elemento?
- `unordered_map` → ¿Qué valor corresponde a esta clave?
- `set` → Elementos únicos + ordenados.
- `map` → Claves → valores + claves ordenadas.

4 Trees

5 Range Queries

5.1 Casos de uso

Cuando el problema te pide:	Entonces usa:	Complejidad
Suma de rango sobre un arreglo que nunca cambia , con muchas consultas (ej. "suma de ventas entre el día l y r ")	Prefix Sums	Build $O(n)$ Query $O(1)$
Suma de rango con actualizaciones puntuales frecuentes (ej. "suma el stock de la posición i , luego consulta un rango")	Fenwick Tree (BIT)	Build $O(n)$ Query/Update $O(\log n)$
Sumar v a todo un rango $[l, r]$ y también consultar la suma de cualquier rango	FenwickRangeUpdate (doble BIT)	Query/Update $O(\log n)$
Range update + range query con operaciones más complejas: min/max en rango, asignar un valor a todo un rango, contar ceros, etc.	Segment Tree + Lazy Propagation	Query/Update $O(\log n)$
Mínimo, máximo o gcd de cualquier rango sobre un arreglo estático (ej. RMQ clásico, LCA) Query $O(1)$	Sparse Table	Build $O(n \log n)$
Mínimo o máximo de rango cuando el arreglo sí cambia con el tiempo	Segment Tree	Query/Update $O(\log n)$
Consultas 2D: cuántos puntos o cuánta suma hay dentro de un rectángulo $[x_1, x_2] \times [y_1, y_2]$	Fenwick 2D	Query/Update $O(\log n \log m)$
Encontrar el k -ésimo elemento, o "menor índice cuya suma acumulada alcanza X " (frecuencias)	Fenwick con lowerBound	$O(\log n)$
Muchas queries de rango offline (las conoces todas de antemano) sobre un arreglo estático o con pocos updates	Mo's Algorithm	$O((n+q)\sqrt{n})$
LCA entre dos nodos, o mínimo en rango sobre un recorrido Euler de un árbol (valores consecutivos difieren en ± 1)	RMQ ± 1 (Euler Tour Sparse Table)	Build $O(n)$ Query $O(1)$
Cuántos elementos en el rango $[l, r]$ están dentro de un rango de valores $[a, b]$, o cuál es el k -ésimo menor en $[l, r]$	Wavelet Tree	Query $O(\log n)$
Queries o updates sobre caminos o subárboles de un árbol (ej. "suma el camino de u a v ")	Heavy-Light Decomposition (+ Segment/Fenwick)	Query/Update $O(\log^2 n)$
Necesitas algo simple de implementar bajo presión, con balance razonable entre update y query, sin memorizar mucha teoría	Sqrt Decomposition	Query/Update $O(\sqrt{n})$

Table 2: Guía rápida: qué estructura de range queries usar según el problema, con su complejidad.

5.2 Prefix Sums

Idea: si un arreglo **no cambia**, puedes precalcular $\text{pref}[i]$ = suma de los primeros i elementos en $O(n)$, una sola vez. Cualquier suma de rango $[l, r]$ se responde restando dos prefijos: $\text{pref}[r+1] - \text{pref}[l]$. Es la base conceptual de todo lo demás en esta sección: Fenwick y Segment Tree existen justamente para soportar lo que Prefix Sums no puede (actualizaciones), a costa de más complejidad de código.

¿Por qué usarlo? Es lo más simple y rápido posible para consultas de rango cuando el arreglo es estático. Cero estructuras de datos elaboradas, cero recursión, solo un arreglo auxiliar y una resta.

Casos de uso:

- Suma de rango sobre un arreglo que nunca se modifica, con muchas queries.

- Suma de submatriz (versión 2D) sobre una matriz estática.
- Aplicar B actualizaciones de rango **offline** y consultar el arreglo final una sola vez (Difference Array).
- Como paso final de un precómputo de teoría de números (criba de ϕ , μ , número de divisores, etc.) cuando el problema pide sumas sobre esos valores en un rango.

Complejidad: construir $O(n)$ (u $O(nm)$ en 2D), consulta $O(1)$. No soporta actualizaciones puntuales eficientes: modificar un elemento y volver a consultar obliga a reconstruir, $O(n)$.

Prefix Sum 1D

```

1 struct PrefixSum1D {
2     vector<long long> pref;
3     int n;
4
5     // CREATE: construye los prefijos a partir de a[0..n-1].
6     PrefixSum1D(vector<long long>& a) {
7         n = a.size();
8         pref.assign(n + 1, 0);
9
10        for (int i = 0; i < n; i++)
11            pref[i + 1] = pref[i] + a[i];
12    } // O(n)
13
14    // READ: suma del rango [l, r] (0-indexado, inclusivo).
15    long long rango(int l, int r) {
16        if (l > r) return 0;
17        return pref[r + 1] - pref[l];
18    } // O(1)
19 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7};
2
3 PrefixSum1D ps(a);
4 // arreglo logico: [5, 2, 9, 1, 7] (posiciones 0..4)
5
6 cout << ps.rango(0, 2);
7 // 5 + 2 + 9 = 16
8
9 cout << ps.rango(1, 4);
10 // 2 + 9 + 1 + 7 = 19
11
12 cout << ps.rango(3, 3);
13 // 1

```

Prefix Sum 2D

```

1 struct PrefixSum2D {
2     vector<vector<long long>> pref;
3     int n, m;
4
5     // CREATE: construye la matriz de prefijos a partir de
6     // a[0..n-1][0..m-1] usando inclusion-exclusion.
7     PrefixSum2D(vector<vector<long long>>& a) {
8         n = a.size();
9         m = a[0].size();
10        pref.assign(n + 1, vector<long long>(m + 1, 0));
11
12        for (int i = 0; i < n; i++)
13            for (int j = 0; j < m; j++)
14                pref[i + 1][j + 1] = a[i][j] + pref[i][j + 1]
15                + pref[i + 1][j] - pref[i][j];
16    } // O(n * m)
17
18    // READ: suma del rectangulo [x1..x2] x [y1..y2] (0-indexado,
19    // inclusivo), usando inclusion-exclusion sobre los prefijos.
20    long long rango(int x1, int y1, int x2, int y2) {
21        return pref[x2 + 1][y2 + 1] - pref[x1][y2 + 1]
22        - pref[x2 + 1][y1] + pref[x1][y1];
23    } // O(1)
24 };

```

Ejemplo de funcionamiento:

```

1 vector<vector<long long>> a = {
2     {1, 2, 3},
3     {4, 5, 6},
4     {7, 8, 9}
5 };
6
7 PrefixSum2D ps(a);
8
9 cout << ps.rango(0, 0, 1, 1);
10 // 1+2+4+5 = 12
11
12 cout << ps.rango(1, 1, 2, 2);
13 // 5+6+8+9 = 28

```

Difference Array (updates de rango offline)

Idea: el truco inverso a Prefix Sums: para sumar v a todo un rango $[l, r]$, marcas $\text{diff}[l] += v$ y $\text{diff}[r+1] -= v$ en $O(1)$. Después de aplicar **todas** las actualizaciones, un solo recorrido con prefijos reconstruye el arreglo final. Sirve solo cuando todas las updates son **offline** (las conoces antes de necesitar consultar); si necesitas consultar *entre* actualizaciones, usa Fenwick-RangeUpdate en su lugar.

```

1 struct DifferenceArray {
2     vector<long long> diff;
3     int n;
4
5     // CREATE: reserva un arreglo de diferencias de tamaño n.
6     DifferenceArray(int n) : n(n) {
7         diff.assign(n + 1, 0);
8     }
9
10    // UPDATE: marca +v en todo el rango [l, r] (0-indexado).
11    void rangoUpdate(int l, int r, long long v) {
12        diff[l] += v;
13        if (r + 1 <= n)
14            diff[r + 1] -= v;
15    }
16
17    // READ: reconstruye el arreglo final tras aplicar todas
18    // las actualizaciones marcadas hasta el momento.
19    vector<long long> aplicar() {
20        vector<long long> a(n);
21        long long acc = 0;
22
23        for (int i = 0; i < n; i++) {
24            acc += diff[i];
25            a[i] = acc;
26        }
27
28        return a;
29    }
30 };
31 // 0(1)
32 // 0(n)

```

Ejemplo de funcionamiento:

```

1 DifferenceArray d(5);
2
3 d.rangoUpdate(1, 3, 4);
4 d.rangoUpdate(0, 4, 1);
5 d.rangoUpdate(2, 2, 10);
6
7 vector<long long> resultado = d.aplicar();
8 // resultado: [1, 5, 15, 5, 1]

```

Aplicación: suma de Euler Totient (ϕ) en un rango

Idea: $\phi(n)$ cuenta cuántos enteros en $[1, n]$ son coprimos con n . Calcular ϕ de un número es teoría de números ($O(\sqrt{n})$ factorizando); pero cuando el problema pide la **suma de $\phi(i)$ para i en un rango $[l, r]$** , con muchas queries, el patrón se resuelve en dos fases: (1) precalculas el arreglo de ϕ una sola vez con una criba (teoría de números), y (2) respondes las queries de rango con Prefix Sums sobre ese arreglo ya calculado (exactamente esta

subsección). Factorizar cada número del rango por separado en cada query sería $O(q\sqrt{n})$ y no escala; precalcular y sumar con prefijos reduce cada query a $O(1)$.

Criba lineal + Prefix Sums (rango $[1, N]$).

```

1 struct PhiRangeSum {
2     vector<long long> phi, pref;
3     int n;
4
5     // CREATE: calcula phi[1..n] con criba lineal y sus prefijos.
6     // phi[i] = cantidad de enteros en [1, i] coprimos con i.
7     PhiRangeSum(int n) : n(n) {
8         phi.assign(n + 1, 0);
9         pref.assign(n + 1, 0);
10
11        vector<int> primos;
12        vector<bool> compuesto(n + 1, false);
13        phi[1] = 1;
14
15        for (int i = 2; i <= n; i++) {
16            if (!compuesto[i]) {
17                primos.push_back(i);
18                phi[i] = i - 1;
19            }
20
21            for (int p : primos) {
22                if ((long long)i * p > n) break;
23
24                compuesto[i * p] = true;
25
26                if (i % p == 0) {
27                    phi[i * p] = phi[i] * p;
28                    break;
29                } else {
30                    phi[i * p] = phi[i] * (p - 1);
31                }
32            }
33        }
34
35        for (int i = 1; i <= n; i++)
36            pref[i] = pref[i - 1] + phi[i];
37    }
38
39    // READ: suma de phi(i) para i en [l, r].
40    long long rango(int l, int r) {
41        if (l > r) return 0;
42        return pref[r] - pref[l - 1];
43    }
44
45    // READ: valor individual de phi(i).
46    long long get(int i) {
47        return phi[i];
48    }
49 };
50 // 0(n)
51 // 0(1)

```

Ejemplo de funcionamiento:

```

1 PhiRangeSum pr(10);
2 // phi(1..10) = [1, 1, 2, 2, 4, 2, 6, 4, 6, 4]
3
4 cout << pr.get(7);
5 // phi(7) = 6 (7 es primo)
6
7 cout << pr.rango(1, 10);
8 // 1+1+2+2+4+2+6+4+6+4 = 32
9
10 cout << pr.rango(4, 8);
11 // 2+4+2+6+4 = 18

```

Criba segmentada (rango $[L, R]$ grande). Si R es muy grande (10^{12}) pero $R - L$ es manejable, no puedes cribar desde 1. En vez de eso, precalculas los primos hasta $\sqrt{R}$, y para cada número del segmento $[L, R]$ le quitas sus factores primos pequeños, aplicando $\phi(n) = n \prod_{p|n} (1 - \frac{1}{p})$. Si al final queda un residuo > 1 , es un único primo grande (mayor a $\sqrt{R}$) que también se aplica a la fórmula.

```

1 struct PhiSegmentedRangeSum {
2     vector<long long> phi, pref;
3     long long L, R;
4
5     // CREATE: calcula phi(i) para cada i en [L, R] factorizando
6     // con

```

```

6 // los primos hasta sqrt(R), y sus prefijos sobre el segmento.
7 PhiSegmentedRangeSum(long long L, long long R) : L(L), R(R) {
8     int lim = (int)sqrt((double)R) + 1;
9
10    vector<int> primos;
11    vector<bool> compuesto(lim + 1, false);
12    for (int i = 2; i <= lim; i++) {
13        if (!compuesto[i]) primos.push_back(i);
14        for (int p : primos) {
15            if ((long long)i * p > lim) break;
16            compuesto[i * p] = true;
17            if (i % p == 0) break;
18        }
19    } // O(sqrt(R) log
    log sqrt(R))
20
21    int sz = R - L + 1;
22    vector<long long> num(sz);
23    phi.assign(sz, 0);
24
25    for (int i = 0; i < sz; i++) {
26        num[i] = L + i;
27        phi[i] = L + i;
28    }
29
30    for (int p : primos) {
31        long long start = ((L + p - 1) / p) * p;
32
33        for (long long j = start; j <= R; j += p) {
34            int idx = j - L;
35            phi[idx] -= phi[idx] / p;
36
37            while (num[idx] % p == 0)
38                num[idx] /= p;
39        }
40    }
41
42    // lo que quede > 1 es un primo grande (> sqrt(R)).
43    for (int i = 0; i < sz; i++)
44        if (num[i] > 1)
45            phi[i] -= phi[i] / num[i];
46
47    pref.assign(sz + 1, 0);
48    for (int i = 0; i < sz; i++)
49        pref[i + 1] = pref[i] + phi[i];
50 } // O((R-L+1) log
    log R + sqrt(R))
51
52 // READ: suma de phi(i) para i en [l, r], con L <= l <= r <=
    R.
53 long long rango(long long l, long long r) {
54     return pref[r - L + 1] - pref[l - L];
55 } // O(1)
56
57 // READ: valor individual de phi(i), con L <= i <= R.
58 long long get(long long i) {
59     return phi[i - L];
60 } // O(1)
61 };

```

Ejemplo de funcionamiento:

```

1 PhiSegmentedRangeSum ps(95, 105);
2 // calcula phi(95), phi(96), ..., phi(105)
3 // sin cribar desde 1
4
5 cout << ps.get(100);
6 // phi(100) = 40
7
8 cout << ps.rango(95, 105);
9 // suma de phi(95) a phi(105)
10
11 cout << ps.rango(100, 102);
12 // phi(100) + phi(101) + phi(102)

```

Cuál variante usar:

Situación	Estructura	Complejidad
$[1, N]$, N chico/mediano	PhiRangeSum	$O(N)$
$[L, R]$ grande, $R - L$ chico	PhiSegmentedRangeSum	$O((R - L) \log \log R + \sqrt{R})$
ϕ cambia entre queries	Fenwick sobre ϕ	$O(\log N)$

Table 3: Variantes para la suma de ϕ .

Regla práctica: el cálculo de ϕ en sí (la criba, lineal o segmentada) es teoría de números y no cambia según la query. Una vez que tienes el arreglo de ϕ listo, trátalo como *cualquier otro arreglo estático*: Prefix Sums

resuelve el resto. No hace falta una estructura especial para ϕ más allá de construir bien el arreglo inicial.

5.3 Árbol de Fenwick (BIT)

Idea: un arreglo normal da suma de prefijo en $O(1)$ pero actualizar un elemento cuesta $O(n)$ (o al revés si guardas prefijos). El Fenwick equilibra ambas a $O(\log n)$ guardando *sumas parciales inteligentes*: la posición i almacena la suma de un bloque de tamaño $i \& (-i)$ (su bit menos significativo) que termina en i . Para consultar un prefijo saltas hacia atrás quitando el bit bajo ($i -= i \& (-i)$); para actualizar subes sumándolo ($i += i \& (-i)$). Cada camino toca a lo más $\log n$ posiciones (un bit por paso). Es 1-indexado.

¿Por qué usarlo? Es mucho más simple y liviano que un Segment Tree cuando solo necesitas sumas (o cualquier operación invertible) de prefijo/rango con actualizaciones puntuales. Menos memoria, menos código y una constante más pequeña.

Casos de uso:

- Suma de prefijo / suma de rango con updates frecuentes.
- Conteo de inversiones en un arreglo.
- Frecuencias acumuladas y búsqueda del k -ésimo elemento.
- Range update + range query cuando no hace falta un Segment Tree con Lazy Propagation.
- Consultas 2D: conteo/suma de puntos dentro de un rectángulo.

Complejidad: construir $O(n)$, update puntual $O(\log n)$, query de prefijo/rango $O(\log n)$, range update + range query $O(\log n)$ y versión 2D $O(\log n \cdot \log m)$.

BIT clásico (Point Update, Range Query)

```

1 struct FenwickTree {
2     vector<long long> t;
3     int n;
4
5     // CREATE: reserva un arbol de tamaño n (1-indexado).
6     FenwickTree(int n) : t(n + 1, 0), n(n) {}
7
8     // CREATE: construye el arbol a partir de un arreglo a[0..n-1]
9     // propagando cada valor a su "padre" una sola vez, en vez de
10    // hacer n updates (que serian O(n log n)).
11    void build(vector<long long> &a) {
12        for (int i = 1; i <= n; i++) {
13            t[i] += a[i - 1];
14
15            int j = i + (i & (-i));
16
17            if (j <= n)
18                t[j] += t[i];
19        }
20    } // O(n)
21
22    // UPDATE: suma v al elemento en la posición i.
23    void update(int i, long long v) {
24        for (; i <= n; i += i & (-i))
25            t[i] += v;
26    } // O(log n)
27
28    // UPDATE: fija el valor absoluto de la posición i en x.
29    void set(int i, long long x) {
30        update(i, x - get(i));
31    } // O(log n)
32
33    // READ: suma de prefijo [1..i].

```



```

34 long long query(int i) {
35     long long s = 0;
36
37     for (; i > 0; i -= i & (-i))
38         s += t[i];
39
40     return s;
41 } // O(log n)
42
43 // READ: suma del rango [l, r].
44 long long rango(int l, int r) {
45     if (l > r)
46         return 0;
47
48     return query(r) - query(l - 1);
49 } // O(log n)
50
51 // READ: valor puntual en la posición i.
52 long long get(int i) {
53     return rango(i, i);
54 } // O(log n)
55
56 // SEARCH: menor índice i tal que query(i) >= objetivo.
57 // Usa binary lifting sobre el árbol.
58 // Requiere que todos los valores sean >= 0.
59 int lowerBound(long long objetivo) {
60     int pos = 0;
61     long long ac = 0;
62
63     int LOG = 0;
64
65     while ((1 << LOG) <= n)
66         LOG++;
67
68     for (int k = LOG - 1; k >= 0; k--) {
69         int nx = pos + (1 << k);
70
71         if (nx <= n && ac + t[nx] < objetivo) {
72             pos = nx;
73             ac += t[nx];
74         }
75     }
76
77     return pos + 1;
78 } // O(log n)
79
80 // DELETE: reinicia el árbol a ceros.
81 void clear() {
82     fill(t.begin(), t.end(), 0);
83 } // O(n)
84 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7};
2
3 FenwickTree bit(5);
4 bit.build(a);
5
6 // arreglo logico:
7 // [5, 2, 9, 1, 7]
8 // posiciones 1..5
9
10 cout << bit.query(3);
11 // 5 + 2 + 9 = 16
12
13 cout << bit.rango(2, 4);
14 // 2 + 9 + 1 = 12
15
16 bit.update(2, 3);
17
18 // arreglo logico:
19 // [5, 5, 9, 1, 7]
20
21 cout << bit.get(2);
22 // 5
23
24 bit.set(4, 10);
25
26 // arreglo logico:
27 // [5, 5, 9, 10, 7]
28
29 cout << bit.lowerBound(15);
30 // menor índice cuyo prefijo alcanza 15
31 // -> índice 3 (5 + 5 + 9 = 19 >= 15)

```

BIT con Range Update + Range Query (Doble BIT)

Idea: con *dos* Fenwick y el truco de diferencias se puede sumar v a todo un rango $[l, r]$ en $O(\log n)$ y también consultar la suma de cualquier rango en $O(\log n)$. Esto permite resolver simultáneamente actualizaciones y consultas de rango.

```

1 struct FenwickRangeUpdate {
2     FenwickTree b1, b2;
3     int n;
4
5     // CREATE: reserva dos arboles auxiliares de tamaño n.
6     FenwickRangeUpdate(int n) : b1(n), b2(n), n(n) {}
7
8     // UPDATE: suma v a todos los elementos del rango [l, r].
9     void rangoUpdate(int l, int r, long long v) {
10         b1.update(l, v);
11         b1.update(r + 1, -v);
12
13         b2.update(l, v * (1 - 1));
14         b2.update(r + 1, -v * r);
15     } // O(log n)
16
17     // READ: suma de prefijo [1..i].
18     long long prefijo(int i) {
19         return b1.query(i) * i - b2.query(i);
20     } // O(log n)
21
22     // READ: suma del rango [l, r].
23     long long rango(int l, int r) {
24         if (l > r)
25             return 0;
26
27         return prefijo(r) - prefijo(l - 1);
28     } // O(log n)
29
30     // READ: valor puntual en la posición i.
31     long long get(int i) {
32         return rango(i, i);
33     } // O(log n)
34 };

```

Ejemplo de funcionamiento:

```

1 FenwickRangeUpdate f(5);
2 // arreglo logico inicial: [0, 0, 0, 0, 0]
3
4 f.rangoUpdate(2, 4, 3);
5 // arreglo logico: [0, 3, 3, 3, 0]
6
7 cout << f.rango(1, 5);
8 // 0+3+3+3+0 = 9
9
10 f.rangoUpdate(1, 3, 2);
11 // arreglo logico: [2, 5, 5, 3, 0]
12
13 cout << f.get(2);
14 // 5

```

BIT 2D (point update, sum de submatriz)

Idea: un BIT dentro de otro BIT. Cada update/query en x dispara un update/query completo en y , dando $O(\log n \cdot \log m)$ en vez de $O(nm)$ por consulta.

```

1 struct Fenwick2D {
2     vector<vector<long long>>> t;
3     int n, m;
4
5     // CREATE: reserva una matriz de arboles de tamaño n x m.
6     Fenwick2D(int n, int m) : t(n + 1, vector<long long>(m + 1, 0)),
7                               n(n), m(m) {}
8
9     // UPDATE: suma v a la celda (x, y).
10    void update(int x, int y, long long v) {
11        for (int i = x; i <= n; i += i & (-i))
12            for (int j = y; j <= m; j += j & (-j))
13                t[i][j] += v;
14    } // O(log n * log m)
15
16    // READ: suma de la submatriz [1..x] x [1..y].
17    long long query(int x, int y) {
18        long long s = 0;

```

```

19     for (int i = x; i > 0; i -= i & (-i))
20         for (int j = y; j > 0; j -= j & (-j))
21             s += t[i][j];
22     return s;
23 }                                     // O(log n * log m)
24 )
25 // READ: suma del rectangulo [x1..x2] x [y1..y2].
26 long long rango(int x1, int y1, int x2, int y2) {
27     return query(x2, y2) - query(x1 - 1, y2)
28         - query(x2, y1 - 1) + query(x1 - 1, y1 - 1);
29 }                                     // O(log n * log m)
30 }

```

Ejemplo de funcionamiento:

```

1 Fenwick2D f(4, 4);
2
3 f.update(1, 1, 5);
4 f.update(2, 3, 7);
5 f.update(4, 4, 2);
6
7 cout << f.rango(1, 1, 2, 3);
8 // 5 + 7 = 12
9
10 cout << f.rango(3, 3, 4, 4);
11 // 2

```

Diferencia con un arreglo de prefijos simple:

Estructura	Construir	Query	Update
Prefijos simples	$O(n)$	$O(1)$	$O(n)$
FenwickTree	$O(n)$	$O(\log n)$	$O(\log n)$
FenwickRangeUpdate	$O(n)$	$O(\log n)$	$O(\log n)^*$
Fenwick2D	$O(nm)$	$O(\log n \log m)$	$O(\log n \log m)$

Table 4: Comparación de complejidades: Fenwick vs. prefijos simples.

*Range update + range query, con la variante de doble BIT.

Regla práctica: si el arreglo cambia con frecuencia y solo necesitas sumas (o cualquier operación invertible) de prefijo o rango, el Fenwick da el mejor balance entre simplicidad de código y velocidad. Si necesitas min/max, operaciones no invertibles, o updates de rango con lazy propagation general, usa Segment Tree en su lugar.

5.4 Sqrt Decomposition

Idea: divide el arreglo en bloques de tamaño $\approx \sqrt{n}$ y guarda un resumen (suma, mínimo, etc.) por bloque. Un update toca solo un bloque; una query de rango toca a lo más dos bloques *parciales* (recorridos elemento por elemento) y varios bloques *completos* en el medio (usados de golpe con su resumen precalculado). Como hay $\approx \sqrt{n}$ bloques y cada bloque tiene $\approx \sqrt{n}$ elementos, todo cuesta $O(\sqrt{n})$: ni tan lento como un arreglo plano, ni tan rápido como Fenwick/Segment Tree, pero mucho más simple de escribir y de adaptar a operaciones raras.

¿Por qué usarlo? Es la estructura más flexible cuando no tienes tiempo de pensar un Segment Tree con lazy propagation correcto: soporta casi cualquier operación (suma, min, max, contar algo con una condición) con el mismo esqueleto de código, aunque sea $O(\sqrt{n})$ en vez de $O(\log n)$. También es la base conceptual de Mo's Algorithm.

Casos de uso:

- Necesitas implementar algo rápido bajo presión, sin memorizar casos de Segment Tree con lazy propagation.

- Operaciones donde min/max cambian de forma que Fenwick no puede manejar (Fenwick solo sirve para operaciones invertibles como la suma).
- n no es tan grande ($\leq 10^5$ – 10^6) y $O(\sqrt{n})$ por operación es suficiente para el límite de tiempo.
- Base para Mo's Algorithm (ver esa sección).

Complejidad: construir $O(n)$, update puntual $O(1)$ (suma) u $O(\sqrt{n})$ (min/max), query de rango $O(\sqrt{n})$, range update + range query $O(\sqrt{n})$ con lazy por bloque.

Sqrt Decomposition clásico (point update, range query — suma)

```

1 struct SqrtDecomposition {
2     vector<long long> a, bloque;
3     int n, tam;
4
5     // CREATE: construye el arreglo en bloques de tamaño ~sqrt(n).
6
7     SqrtDecomposition(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tam = max(1, (int)sqrt((double)n));
11
12        bloque.assign((n + tam - 1) / tam, 0);
13
14        for (int i = 0; i < n; i++)
15            bloque[i / tam] += a[i];
16    }                                     // O(n)
17
18    // UPDATE: suma v al elemento en la posición i.
19    void update(int i, long long v) {
20        a[i] += v;
21        bloque[i / tam] += v;
22    }                                     // O(1)
23
24    // READ: suma del rango [l, r] (0-indexado, inclusivo).
25    long long rango(int l, int r) {
26        long long s = 0;
27        int bl = l / tam, br = r / tam;
28
29        if (bl == br) {
30            for (int i = l; i <= r; i++)
31                s += a[i];
32            return s;
33        }
34
35        for (int i = l; i < (bl + 1) * tam; i++)
36            s += a[i];
37
38        for (int b = bl + 1; b < br; b++)
39            s += bloque[b];
40
41        for (int i = br * tam; i <= r; i++)
42            s += a[i];
43
44        return s;
45    }                                     // O(sqrt(n))
46 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3, 8, 4};
2
3 SqrtDecomposition sd(a);
4 // arreglo logico: [5, 2, 9, 1, 7, 3, 8, 4] (posiciones 0..7)
5 // tam = 2 -> bloques: [5+2, 9+1, 7+3, 8+4] = [7, 10, 10, 12]
6
7 cout << sd.rango(1, 5);
8 // 2 + 9 + 1 + 7 + 3 = 22
9
10 sd.update(3, 10);
11 // arreglo logico: [5, 2, 9, 11, 7, 3, 8, 4]
12
13 cout << sd.rango(1, 5);
14 // 2 + 9 + 11 + 7 + 3 = 32

```

Sqrt Decomposition con range update + range query (lazy por bloque)

Idea: igual que arriba, pero cada bloque guarda un valor lazy pendiente que representa “sumar v a todo el bloque”

sin tocar cada elemento individualmente. Solo se aplica a los elementos reales cuando un update o query **parcial** necesita tocar ese bloque específico.

```

1 struct SqrtRangeUpdate {
2     vector<long long> a, bloqueSum, lazy;
3     int n, tam;
4
5     // CREATE: construye el arreglo en bloques de tamaño ~sqrt(n).
6
7     SqrtRangeUpdate(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tam = max(1, (int)sqrt((double)n));
11
12        int nb = (n + tam - 1) / tam;
13        bloqueSum.assign(nb, 0);
14        lazy.assign(nb, 0);
15
16        for (int i = 0; i < n; i++)
17            bloqueSum[i / tam] += a[i]; // O(n)
18    }
19
20    // READ auxiliar: cantidad de elementos reales del bloque b.
21    int tamBloque(int b) {
22        return min(n, (b + 1) * tam) - b * tam; // O(1)
23    }
24
25    // UPDATE: suma v a todo el rango [l, r].
26    void rangoUpdate(int l, int r, long long v) {
27        int bl = l / tam, br = r / tam;
28
29        if (bl == br) {
30            for (int i = l; i <= r; i++)
31                a[i] += v;
32            bloqueSum[bl] += v * (r - l + 1);
33            return;
34        }
35
36        for (int i = l; i < (bl + 1) * tam; i++)
37            a[i] += v;
38        bloqueSum[bl] += v * ((bl + 1) * tam - l);
39
40        for (int b = bl + 1; b < br; b++)
41            lazy[b] += v;
42
43        for (int i = br * tam; i <= r; i++)
44            a[i] += v;
45        bloqueSum[br] += v * (r - br * tam + 1); // O(sqrt(n))
46    }
47
48    // READ: suma del rango [l, r].
49    long long rango(int l, int r) {
50        long long s = 0;
51        int bl = l / tam, br = r / tam;
52
53        if (bl == br) {
54            for (int i = l; i <= r; i++)
55                s += a[i] + lazy[bl];
56            return s;
57        }
58
59        for (int i = l; i < (bl + 1) * tam; i++)
60            s += a[i] + lazy[bl];
61
62        for (int b = bl + 1; b < br; b++)
63            s += bloqueSum[b] + lazy[b] * tamBloque(b);
64
65        for (int i = br * tam; i <= r; i++)
66            s += a[i] + lazy[br];
67
68        return s; // O(sqrt(n))
69    }
70 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {1, 1, 1, 1, 1, 1};
2
3 SqrtRangeUpdate sr(a);
4 // tam = 2 -> bloques: [1+1, 1+1, 1+1] = [2, 2, 2]
5
6 sr.rangoUpdate(1, 4, 10);
7 // arreglo logico "efectivo": [1, 11, 11, 11, 11, 1]
8
9 cout << sr.rango(0, 5);
10 // 1+11+11+11+11+1 = 46
11
12 cout << sr.rango(2, 3);
13 // 11+11 = 22

```

Variante para mínimo / máximo (point update, range query)

Idea: a diferencia de Fenwick (que solo sirve para operaciones *invertibles* como la suma), Sqrt Decomposition soporta min/max con el mismo esqueleto: la única diferencia es que un update ya no puede ajustar el resumen del bloque en $O(1)$, porque quitar el valor viejo no dice si sigue habiendo otro elemento igual de chico. Hay que **reconstruir el mínimo de todo el bloque**, lo cual sigue siendo $O(\sqrt{n})$.

```

1 struct SqrtMinQuery {
2     vector<long long> a, bloqueMin;
3     int n, tam;
4
5     // CREATE: construye el arreglo en bloques de tamaño ~sqrt(n).
6
7     SqrtMinQuery(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tam = max(1, (int)sqrt((double)n));
11
12        bloqueMin.assign((n + tam - 1) / tam, LLONG_MAX);
13
14        for (int i = 0; i < n; i++)
15            bloqueMin[i / tam] = min(bloqueMin[i / tam], a[i]); // O(n)
16    }
17
18    // UPDATE: fija el valor absoluto de la posición i en x.
19    // Reconstruye el mínimo de todo el bloque, ya que a
20    // diferencia
21    // de la suma, el mínimo no se puede ajustar incrementalmente.
22
23    void set(int i, long long x) {
24        a[i] = x;
25
26        int b = i / tam;
27        long long mn = LLONG_MAX;
28
29        for (int j = b * tam; j < min(n, (b + 1) * tam); j++)
30            mn = min(mn, a[j]);
31
32        bloqueMin[b] = mn; // O(sqrt(n))
33    }
34
35    // READ: mínimo del rango [l, r].
36    long long rango(int l, int r) {
37        long long mn = LLONG_MAX;
38        int bl = l / tam, br = r / tam;
39
40        if (bl == br) {
41            for (int i = l; i <= r; i++)
42                mn = min(mn, a[i]);
43            return mn;
44        }
45
46        for (int i = l; i < (bl + 1) * tam; i++)
47            mn = min(mn, a[i]);
48
49        for (int b = bl + 1; b < br; b++)
50            mn = min(mn, bloqueMin[b]);
51
52        for (int i = br * tam; i <= r; i++)
53            mn = min(mn, a[i]);
54
55        return mn; // O(sqrt(n))
56    }
57 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3, 8, 4};
2
3 SqrtMinQuery sm(a);
4 // tam = 2 -> bloques min: [2, 1, 3, 4]
5
6 cout << sm.rango(0, 4);
7 // min(5,2,9,1,7) = 1
8
9 sm.set(3, 100);
10 // arreglo logico: [5, 2, 9, 100, 7, 3, 8, 4]
11
12 cout << sm.rango(0, 4);
13 // min(5,2,9,100,7) = 2

```

Comparación con Fenwick y Segment Tree:

Estructura	Update	Query	Min/Max?
Sqrt Decomposition	$O(1) - O(\sqrt{n})$	$O(\sqrt{n})$	Sí
FenwickTree	$O(\log n)$	$O(\log n)$	No
Segment Tree	$O(\log n)$	$O(\log n)$	Sí

Table 5: Sqrt Decomposition vs. Fenwick vs. Segment Tree.

Regla práctica: usa Sqrt Decomposition cuando quieras algo simple y flexible de escribir rápido, sobre todo si la operación no es una suma (Fenwick no sirve) y no quieres arriesgarte con un Segment Tree + Lazy Propagation bajo presión de tiempo. Si n es muy grande o el límite de tiempo es ajustado, $O(\log n)$ de Segment Tree gana; si la operación es solo suma, Fenwick es más simple y más rápido.

5.5 Sparse Table (RMQ)

Idea: responde consultas de rango en $O(1)$ sobre un arreglo que **no cambia**, precalculando $sp[k][i] = \text{el resultado de combinar el bloque que empieza en } i \text{ y tiene largo } 2^k$. La clave está en la consulta: para el rango $[l, r]$ tomas $k = \lfloor \log_2(r - l + 1) \rfloor$ y lo cubres con *dos* bloques de largo 2^k —uno pegado a l y otro pegado a r — que **se solapan**. Si la operación es *idempotente* ($f(x, x) = x$, como min, max o gcd), contar el solape dos veces no cambia el resultado, así que la consulta son solo dos lecturas y una combinación. Para operaciones **no idempotentes** (como suma o xor) hace falta la variante *Disjoint Sparse Table*, que evita el solape por construcción.

¿Por qué usarlo? Es la estructura más rápida posible para consultas de rango ($O(1)$ por query) cuando el arreglo es estático. A cambio de esa velocidad, no soporta actualizaciones: reconstruir cuesta $O(n \log n)$, así que si el arreglo cambia, conviene un Segment Tree.

Casos de uso:

- RMQ clásico: mínimo o máximo de cualquier rango, arreglo estático.
- gcd o AND/OR de rango sobre un arreglo estático.
- LCA vía Euler Tour + RMQ ± 1 .
- Suma, xor o cualquier operación no idempotente de rango estático, en $O(1)$, usando Disjoint Sparse Table.
- Cualquier problema con muchísimas queries offline sobre datos que nunca se modifican.

Complejidad: construir $O(n \log n)$, consulta $O(1)$. (Si el arreglo *cambia*, usa Segment Tree: $O(\log n)$ por operación.)

Sparse Table clásico (operaciones idempotentes: min, max, gcd)

```
1 struct SparseTable {
2     vector<vector<long long>>> sp;
3     vector<int> lg;
4     int n;
5
6     // CREATE: precalcula la tabla a partir de un arreglo a[0..n-1].
7     // sp[k][i] = mínimo del bloque [i, i + 2^k - 1].
8     SparseTable(vector<long long>& a) {
```

```
9         n = a.size();
10
11         lg.assign(n + 1, 0);
12         for (int i = 2; i <= n; i++)
13             lg[i] = lg[i / 2] + 1;
14
15         int K = lg[n] + 1;
16         sp.assign(K, vector<long long>(n));
17         sp[0] = a;
18
19         for (int k = 1; k < K; k++)
20             for (int i = 0; i + (1 << k) <= n; i++)
21                 sp[k][i] = min(sp[k - 1][i],
22                               sp[k - 1][i + (1 << (k - 1))]);
23     }
24
25     // READ: mínimo del rango [l, r] (0-indexado, inclusivo).
26     // Cubre [l, r] con dos bloques de largo 2^k que se solapan;
27     // como min es idempotente, el solape no afecta el resultado.
28     long long query(int l, int r) {
29         int k = lg[r - l + 1];
30         return min(sp[k][l], sp[k][r - (1 << k) + 1]);
31     }
32 };
```

Ejemplo de funcionamiento:

```
1 vector<long long> a = {5, 2, 9, 1, 7, 3};
2
3 SparseTable st(a);
4 // arreglo lógico: [5, 2, 9, 1, 7, 3] (posiciones 0..5)
5
6 cout << st.query(0, 2);
7 // min(5, 2, 9) = 2
8
9 cout << st.query(2, 5);
10 // min(9, 1, 7, 3) = 1
11
12 cout << st.query(4, 4);
13 // min(7) = 7
```

Nota: para max o gcd solo cambia `min(...)` por `max(...)` o `__gcd(...)` en el constructor y en `query`. Si necesitas saber *qué índice* alcanza el mínimo (no solo el valor), guarda índices en vez de valores y compara `a[sp[k][i]]` en lugar de `sp[k][i]`.

Disjoint Sparse Table (operaciones no idempotentes: suma, xor)

Idea: en vez de dos bloques que se solapan, cada nivel k particiona el arreglo en bloques de largo 2^{k+1} y precalcula, para cada bloque, un prefijo hacia la izquierda desde su punto medio y un prefijo hacia la derecha desde ese mismo punto medio. Toda consulta $[l, r]$ cae exactamente en un nivel donde l y r quedan en mitades distintas del mismo bloque (el nivel lo da el bit más alto donde difieren l y r), así que se combina sin solapar y sirve para operaciones **no idempotentes** como la suma.

```
1 struct DisjointSparseTable {
2     vector<vector<long long>>> pre;
3     vector<long long> a;
4     int n, LOG;
5
6     // CREATE: precalcula prefijos izquierda/derecha por nivel,
7     // partiendo el arreglo en bloques de tamaño 2^(k+1).
8     DisjointSparseTable(vector<long long>& arr) {
9         a = arr;
10        n = a.size();
11        LOG = 1;
12        while ((1 << LOG) < n) LOG++;
13        LOG++;
14
15        pre.assign(LOG, vector<long long>(n));
16
17        for (int k = 0; k < LOG; k++) {
18            int block = 1 << (k + 1);
19
20            for (int start = 0; start < n; start += block) {
21                int mid = min(start + block / 2, n);
22                int end = min(start + block, n);
23
24                if (mid > n) continue;
25            }
```

```

26 // prefijo hacia la izquierda desde mid-1
27 if (mid - 1 >= start) {
28     pre[k][mid - 1] = a[mid - 1];
29     for (int i = mid - 2; i >= start; i--)
30         pre[k][i] = a[i] + pre[k][i + 1];
31 }
32
33 // prefijo hacia la derecha desde mid
34 if (mid < end) {
35     pre[k][mid] = a[mid];
36     for (int i = mid + 1; i < end; i++)
37         pre[k][i] = pre[k][i - 1] + a[i];
38 }
39 }
40 }
41 // O(n log n)
42
43 // READ: suma del rango [l, r] (0-indexado, inclusivo).
44 // El nivel k lo da el bit mas alto donde l y r difieren:
45 // ahi es donde el bloque se parte en dos mitades disjuntas.
46 long long query(int l, int r) {
47     if (l == r) return a[l];
48
49     int k = 31 - __builtin_clz(1 ^ r);
50     return pre[k][l] + pre[k][r];
51 }
52 // O(1)
53 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3};
2
3 DisjointSparseTable dst(a);
4 // arreglo logico: [5, 2, 9, 1, 7, 3] (posiciones 0..5)
5
6 cout << dst.query(0, 2);
7 // 5 + 2 + 9 = 16
8
9 cout << dst.query(2, 5);
10 // 9 + 1 + 7 + 3 = 20
11
12 cout << dst.query(1, 1);
13 // 2

```

Diferencia entre las dos variantes:

Estructura	Construir	Query	Updates
Sparse Table (min/max/gcd)	$O(n \log n)$	$O(1)$	No
Disjoint Sparse Table (suma/xor)	$O(n \log n)$	$O(1)$	No
Segment Tree	$O(n)$	$O(\log n)$	Sí, $O(\log n)$

Table 6: Sparse Table vs. Disjoint Sparse Table vs. Segment Tree.

Regla práctica: si el arreglo **no cambia**, usa Sparse Table para min/max/gcd, o Disjoint Sparse Table si necesitas suma, xor, o cualquier operación no idempotente. En cuanto el arreglo **cambia** —aunque sea una sola actualización— reconstruir en $O(n \log n)$ deja de valer la pena y conviene un Segment Tree.

5.6 Segment Tree

Idea: un árbol binario implícito (guardado en un arreglo) donde cada nodo representa el resultado combinado de un rango del arreglo original. La raíz cubre todo $[0, n-1]$; cada nodo se parte en dos mitades hasta llegar a las hojas, que son elementos individuales. Un update baja por el árbol tocando $O(\log n)$ nodos (uno por nivel); una query de rango se descompone en a lo más $O(\log n)$ nodos que cubren exactamente el rango pedido, sin necesidad de bloques parciales como en Sqrt Decomposition. A diferencia de Fenwick, sirve para **cualquier** operación asociativa (min, max, gcd, and, or, no solo sumas), y con Lazy Propagation soporta actualizaciones de rango completo.

¿Por qué usarlo? Es la estructura más versátil de esta sección: $O(\log n)$ tanto para update como para query, funciona con casi cualquier operación asociativa,

y con lazy propagation resuelve range update + range query para operaciones que Fenwick no puede (min, max, asignar un valor a todo un rango, etc.).

Casos de uso:

- Min/max/gcd de rango con actualizaciones (a diferencia de Sparse Table, que exige arreglo estático).
- Range update + range query con operaciones no invertibles: asignar un valor a todo un rango, min/max con suma en rango, etc.
- Cualquier operación asociativa que no sea una suma simple (donde Fenwick ya alcanzaría y sería más simple).
- Base para variantes más avanzadas: Segment Tree 2D, Merge Sort Tree, Persistent Segment Tree, Segment Tree Beats.

Complejidad: construir $O(n)$, update puntual $O(\log n)$, query de rango $O(\log n)$, range update + range query con lazy propagation $O(\log n)$.

Segment Tree clásico (point update, range query — suma)

```

1 struct SegmentTree {
2     vector<long long> tree, a;
3     int n;
4
5     // CREATE: reserva el arbol (tamano 4n es cota segura) y
6     // construye recursivamente a partir de a[0..n-1].
7     SegmentTree(vector<long long>& arr) {
8         a = arr;
9         n = a.size();
10        tree.assign(4 * n, 0);
11        if (n > 0)
12            build(1, 0, n - 1);
13    }
14    // O(n)
15
16    void build(int node, int l, int r) {
17        if (l == r) {
18            tree[node] = a[l];
19            return;
20        }
21
22        int mid = (l + r) / 2;
23        build(2 * node, l, mid);
24        build(2 * node + 1, mid + 1, r);
25        tree[node] = tree[2 * node] + tree[2 * node + 1];
26    }
27
28    void updateUtil(int node, int l, int r, int i, long long x) {
29        if (l == r) {
30            tree[node] = x;
31            return;
32        }
33
34        int mid = (l + r) / 2;
35        if (i <= mid)
36            updateUtil(2 * node, l, mid, i, x);
37        else
38            updateUtil(2 * node + 1, mid + 1, r, i, x);
39
40        tree[node] = tree[2 * node] + tree[2 * node + 1];
41    }
42
43    // UPDATE: fija el valor absoluto de la posicion i en x.
44    void update(int i, long long x) {
45        updateUtil(1, 0, n - 1, i, x);
46    }
47    // O(log n)
48
49    long long queryUtil(int node, int l, int r, int ql, int qr) {
50        if (qr < l || r < ql)
51            return 0;
52        // neutro para
53        // suma
54
55        if (ql <= l && r <= qr)
56            return tree[node];
57
58        int mid = (l + r) / 2;

```



```

55     return queryUtil(2 * node, 1, mid, ql, qr)
56         + queryUtil(2 * node + 1, mid + 1, r, ql, qr);
57 }
58
59 // READ: suma del rango [l, r] (0-indexado, inclusivo).
60 long long rango(int l, int r) {
61     return queryUtil(1, 0, n - 1, l, r);
62 }
63 // O(log n)
};

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7};
2
3 SegmentTree st(a);
4 // arreglo logico: [5, 2, 9, 1, 7] (posiciones 0..4)
5
6 cout << st.rango(1, 3);
7 // 2 + 9 + 1 = 12
8
9 st.update(2, 100);
10 // arreglo logico: [5, 2, 100, 1, 7]
11
12 cout << st.rango(1, 3);
13 // 2 + 100 + 1 = 103

```

Nota: para min/max/gcd solo cambia el neutro del caso base (LLONG_MAX para min, 0 para gcd, etc.) y el operador de combinación (min(...), __gcd(...)) en build, updateUtil y queryUtil.

Segment Tree con Lazy Propagation (range update + range query — suma)

Idea: cada nodo guarda un valor lazy pendiente que representa “sumar v a todo este rango” sin aplicarlo todavía a los hijos. Se aplica (*se propaga*) solo cuando hace falta bajar más en el árbol para otra operación.

```

1 struct SegmentTreeLazyAdd {
2     vector<long long> tree, lazy, a;
3     int n;
4
5     // CREATE: reserva arbol + lazy, construye a partir de a[0..n-1].
6     SegmentTreeLazyAdd(vector<long long>& arr) {
7         a = arr;
8         n = a.size();
9         tree.assign(4 * n, 0);
10        lazy.assign(4 * n, 0);
11        if (n > 0)
12            build(1, 0, n - 1);
13    } // O(n)
14
15    void build(int node, int l, int r) {
16        if (l == r) {
17            tree[node] = a[l];
18            return;
19        }
20
21        int mid = (l + r) / 2;
22        build(2 * node, l, mid);
23        build(2 * node + 1, mid + 1, r);
24        tree[node] = tree[2 * node] + tree[2 * node + 1];
25    }
26
27    // Aplica el lazy pendiente de este nodo y lo empuja a los hijos.
28    void propagar(int node, int l, int r) {
29        if (lazy[node] == 0)
30            return;
31
32        tree[node] += lazy[node] * (r - l + 1);
33
34        if (l != r) {
35            lazy[2 * node] += lazy[node];
36            lazy[2 * node + 1] += lazy[node];
37        }
38
39        lazy[node] = 0;
40    }
41
42    void rangoUpdateUtil(int node, int l, int r, int ql, int qr,
43        long long v) {
44        propagar(node, l, r);
45
46        if (qr < l || r < ql)
47            return;

```

```

48        if (ql <= l && r <= qr) {
49            lazy[node] += v;
50            propagar(node, l, r);
51            return;
52        }
53
54        int mid = (l + r) / 2;
55        rangoUpdateUtil(2 * node, l, mid, ql, qr, v);
56        rangoUpdateUtil(2 * node + 1, mid + 1, r, ql, qr, v);
57        tree[node] = tree[2 * node] + tree[2 * node + 1];
58    }
59
60    // UPDATE: suma v a todo el rango [l, r].
61    void rangoUpdate(int l, int r, long long v) {
62        rangoUpdateUtil(1, 0, n - 1, l, r, v);
63    } // O(log n)
64
65    long long queryUtil(int node, int l, int r, int ql, int qr) {
66        propagar(node, l, r);
67
68        if (qr < l || r < ql)
69            return 0;
70
71        if (ql <= l && r <= qr)
72            return tree[node];
73
74        int mid = (l + r) / 2;
75        return queryUtil(2 * node, l, mid, ql, qr)
76            + queryUtil(2 * node + 1, mid + 1, r, ql, qr);
77    }
78
79    // READ: suma del rango [l, r].
80    long long rango(int l, int r) {
81        return queryUtil(1, 0, n - 1, l, r);
82    } // O(log n)
83
84 };

```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {1, 1, 1, 1, 1, 1};
2
3 SegmentTreeLazyAdd st(a);
4
5 st.rangoUpdate(1, 4, 10);
6 // arreglo logico efectivo: [1, 11, 11, 11, 11, 1]
7
8 cout << st.rango(0, 5);
9 // 1+11+11+11+11+1 = 46
10
11 cout << st.rango(2, 3);
12 // 11+11 = 22

```

Lazy Propagation con asignación (range assign + range query — mínimo)

Idea: en vez de *sumar* un valor al rango, lo *reemplaza* por completo (asigna x a todo $[l, r]$). El lazy aquí no se acumula como en la suma: el último assign pisa a cualquier assign anterior pendiente, así que se necesita una bandera aparte para distinguir “no hay lazy pendiente” de “hay que asignar 0”.

```

1 struct SegmentTreeAssignMin {
2     vector<long long> tree, lazy, a;
3     vector<bool> tieneLazy;
4     int n;
5
6     // CREATE: reserva arbol + lazy + banderas, construye desde a.
7
8     SegmentTreeAssignMin(vector<long long>& arr) {
9         a = arr;
10        n = a.size();
11        tree.assign(4 * n, LLONG_MAX);
12        lazy.assign(4 * n, 0);
13        tieneLazy.assign(4 * n, false);
14        if (n > 0)
15            build(1, 0, n - 1);
16    } // O(n)
17
18    void build(int node, int l, int r) {
19        if (l == r) {
20            tree[node] = a[l];
21            return;
22        }
23
24        int mid = (l + r) / 2;
25        build(2 * node, l, mid);

```



```

25     build(2 * node + 1, mid + 1, r);
26     tree[node] = min(tree[2 * node], tree[2 * node + 1]);
27 }
28
29 // Empuja el assign pendiente de este nodo hacia sus hijos.
30 void propagar(int node) {
31     if (!tieneLazy[node])
32         return;
33
34     for (int hijo : {2 * node, 2 * node + 1}) {
35         tree[hijo] = lazy[node];
36         lazy[hijo] = lazy[node];
37         tieneLazy[hijo] = true;
38     }
39
40     tieneLazy[node] = false;
41 }
42
43 void rangoAssignUtil(int node, int l, int r, int ql, int qr,
44                     long long x) {
45     if (qr < l || r < ql)
46         return;
47
48     if (ql <= l && r <= qr) {
49         tree[node] = x;
50         lazy[node] = x;
51         tieneLazy[node] = true;
52         return;
53     }
54
55     propagar(node);
56     int mid = (l + r) / 2;
57     rangoAssignUtil(2 * node, l, mid, ql, qr, x);
58     rangoAssignUtil(2 * node + 1, mid + 1, r, ql, qr, x);
59     tree[node] = min(tree[2 * node], tree[2 * node + 1]);
60 }
61
62 // UPDATE: asigna x a todo el rango [l, r].
63 void rangoAssign(int l, int r, long long x) {
64     rangoAssignUtil(1, 0, n - 1, l, r, x);
65 }
66 // O(log n)
67
68 long long queryUtil(int node, int l, int r, int ql, int qr) {
69     if (qr < l || r < ql)
70         return LLONG_MAX;
71
72     if (ql <= l && r <= qr)
73         return tree[node];
74
75     propagar(node);
76     int mid = (l + r) / 2;
77     return min(queryUtil(2 * node, l, mid, ql, qr),
78               queryUtil(2 * node + 1, mid + 1, r, ql, qr));
79 }
80
81 // READ: mínimo del rango [l, r].
82 long long rango(int l, int r) {
83     return queryUtil(1, 0, n - 1, l, r);
84 }
85 // O(log n)
86 };
```

Ejemplo de funcionamiento:

```

1 vector<long long> a = {5, 2, 9, 1, 7, 3};
2
3 SegmentTreeAssignMin st(a);
4
5 cout << st.rango(0, 5);
6 // min(5,2,9,1,7,3) = 1
7
8 st.rangoAssign(0, 2, 100);
9 // arreglo logico efectivo: [100, 100, 100, 1, 7, 3]
10
11 cout << st.rango(0, 2);
12 // min(100,100,100) = 100
13
14 cout << st.rango(0, 5);
15 // min(100,100,100,1,7,3) = 1
```

Comparación con las otras estructuras de rango:

Estructura	Update	Query	Min/Max?
Segment Tree	$O(\log n)$	$O(\log n)$	Sí
Segment Tree + Lazy	$O(\log n)$ range	$O(\log n)$ range	Sí
FenwickTree	$O(\log n)$ punto	$O(\log n)$ rango	No
Sqrt Decomposition	$O(1) - O(\sqrt{n})$	$O(\sqrt{n})$	Sí

Table 7: Segment Tree vs. Fenwick vs. Sqrt Decomposition.

Regla práctica: usa Segment Tree cuando necesites min/max/gcd (operaciones no invertibles que Fenwick no soporta), o cuando necesites range update + range query con una operación que no sea suma simple. Si la operación es solo suma y no necesitas range update, Fenwick es más simple y más rápido en la práctica (menor constante). Si no confías en escribir el lazy propagation correcto bajo presión, Sqrt Decomposition es una alternativa más simple a costa de $O(\sqrt{n})$ en vez de $O(\log n)$.

5.7 RMQ (Range Minimum Query)

Idea: RMQ es el problema general de "dado un arreglo, encuentra el mínimo en cualquier rango $[l, r]$ ". Ya tienes las dos estructuras que lo resuelven: **Sparse Table** si el arreglo es estático ($O(1)$ por query) y **Segment Tree** si cambia ($O(\log n)$ por query). Esta sección no repite esas estructuras, sino que muestra las dos aplicaciones clásicas donde un problema que *no parece* de rango se puede **reformular** como RMQ sobre un arreglo auxiliar: LCA en árboles, y LCP entre sufijos arbitrarios de un string.

¿Por qué usarlo? Reconocer cuándo un problema es "RMQ disfrazado" es lo que realmente separa resolver estos problemas en $O(1) - O(\log n)$ de resolverlos con fuerza bruta $O(n)$ por consulta. El patrón se repite: construyes un arreglo auxiliar donde la respuesta a tu pregunta real equivale al mínimo de un rango, y le aplicas Sparse Table o Segment Tree normal.

Casos de uso:

- LCA (ancestro común más bajo) de dos nodos en un árbol, con muchas queries: Euler Tour + RMQ.
- LCP (prefijo común más largo) entre dos sufijos *cualquiera* de un string (no necesariamente consecutivos en el Suffix Array): Suffix Array + Kasai + RMQ sobre el LCP Array.
- Cualquier problema donde la respuesta se pueda expresar como "el mínimo de algo" sobre un rango de índices auxiliares.

Complejidad: en ambas aplicaciones, el precómputo (Euler Tour o Suffix Array + Kasai) es $O(n)$ a $O(n \log^2 n)$ según el método de Suffix Array usado; una vez armado el arreglo auxiliar, el RMQ es $O(n \log n)$ de build y $O(1)$ por query con Sparse Table (ambos casos son estáticos: el árbol y el string no cambian).

LCA vía Euler Tour + RMQ (Sparse Table)

Idea: recorres el árbol con un Euler Tour, anotando el nodo visitado cada vez que **entras o vuelves** a él (arreglo de tamaño $2n - 1$) junto con su profundidad. El LCA de u y v es el nodo de **menor profundidad** que aparece entre la primera aparición de u y la primera aparición de v en ese recorrido — exactamente un RMQ (por profundidad) sobre el arreglo del Euler Tour.

```

1 struct LCA {
2     vector<int> euler, primeraApar, profundidad;
3     vector<vector<int>> sp; // sp[k][i] = nodo con menor
4                             profundidad
5     vector<int> lg;
6
7     // CREATE: construye el Euler Tour del arbol (arr. de tamaño
```

```

7 // 2n-1 y la Sparse Table sobre profundidad, para RMQ O(1).
8 LCA(vector<vector<int>>& adj, int raiz, int n) {
9     primeraApar.assign(n, -1);
10    profundidad.assign(n, 0);
11    euler.reserve(2 * n);
12
13    dfs(adj, raiz, -1, 0);
14    build();
15
16    // O(n log n)
17
18    void dfs(vector<vector<int>>& adj, int u, int p, int d) {
19        primeraApar[u] = euler.size();
20        profundidad[u] = d;
21        euler.push_back(u);
22
23        for (int v : adj[u]) {
24            if (v != p) {
25                dfs(adj, v, u, d + 1);
26                euler.push_back(u);
27            }
28        }
29    }
30
31    void build() {
32        int m = euler.size();
33        lg.assign(m + 1, 0);
34        for (int i = 2; i <= m; i++)
35            lg[i] = lg[i / 2] + 1;
36
37        int K = lg[m] + 1;
38        sp.assign(K, vector<int>(m));
39        sp[0] = euler;
40
41        for (int k = 1; k < K; k++)
42            for (int i = 0; i + (1 << k) <= m; i++) {
43                int l = sp[k - 1][i];
44                int r = sp[k - 1][i + (1 << (k - 1))];
45                sp[k][i] = profundidad[l] < profundidad[r] ? l : r;
46            }
47
48        // READ: LCA de u y v.
49        int query(int u, int v) {
50            int l = primeraApar[u], r = primeraApar[v];
51            if (l > r) swap(l, r);
52
53            int k = lg[r - l + 1];
54            int a = sp[k][l], b = sp[k][r - (1 << k) + 1];
55            return profundidad[a] < profundidad[b] ? a : b;
56        }
57    };

```

Ejemplo de funcionamiento:

```

1 // arbol:      0
2 //           / \
3 //          1  2
4 //         / \
5 //        3  4
6
7 vector<vector<int>> adj(5);
8 adj[0] = {1, 2};
9 adj[1] = {0, 3, 4};
10 adj[2] = {0};
11 adj[3] = {1};
12 adj[4] = {1};
13
14 LCA lca(adj, 0, 5);
15
16 cout << lca.query(3, 4);
17 // 1 (padre comun de 3 y 4)
18
19 cout << lca.query(3, 2);
20 // 0 (raiz)
21
22 cout << lca.query(1, 4);
23 // 1 (uno es ancestro del otro)

```

LCP entre dos sufijos arbitrarios (Suffix Array + Kasai + RMQ)

Idea: el LCP Array (via Kasai) solo te da el LCP entre sufijos **consecutivos** en el Suffix Array. Para el LCP entre dos sufijos **cualquiera** i, j : los ubicas por su rango en el SA ($rnk[i], rnk[j]$) y el LCP entre ambos es el **mínimo** del LCP Array en el rango entre esas dos posiciones —otra vez RMQ, esta vez sobre el LCP Array.

```

1 struct SuffixLCPQuery {
2     vector<int> sa, rnk, lcp;
3     vector<vector<int>> sp;
4     vector<int> lg;
5     int n;
6
7     // CREATE: construye SA, LCP (Kasai) y Sparse Table sobre lcp,
8     // todo en un solo paso.
9     SuffixLCPQuery(const string& s) {
10         n = s.size();
11         sa = buildSuffixArray(s);
12         lcp = buildLCPArray(s, sa);
13
14         rnk.assign(n, 0);
15         for (int i = 0; i < n; i++)
16             rnk[sa[i]] = i;
17
18         buildSparse();
19     }
20
21     // O(n log^2 n)
22
23     void buildSparse() {
24         lg.assign(n + 1, 0);
25         for (int i = 2; i <= n; i++)
26             lg[i] = lg[i / 2] + 1;
27
28         int K = lg[n] + 1;
29         sp.assign(K, vector<int>(n));
30         sp[0] = lcp;
31
32         for (int k = 1; k < K; k++)
33             for (int i = 0; i + (1 << k) <= n; i++)
34                 sp[k][i] = min(sp[k - 1][i], sp[k - 1][i + (1 << (k - 1))]);
35     }
36
37     // READ: LCP entre el sufijo que empieza en i y el que
38     // empieza en j (0-indexado sobre el string original).
39     int lcpEntreSufijos(int i, int j) {
40         if (i == j) return n - i;
41
42         int a = rnk[i], b = rnk[j];
43         if (a > b) swap(a, b);
44
45         // el mínimo va de [a+1, b] en el LCP array
46         int k = lg[b - a];
47         return min(sp[k][a + 1], sp[k][b - (1 << k) + 1]);
48     }
49 };

```

Ejemplo de funcionamiento:

```

1 SuffixLCPQuery slq("banana");
2
3 cout << slq.lcpEntreSufijos(1, 3);
4 // sufijo desde 1: "anana"
5 // sufijo desde 3: "ana"
6 // LCP = "ana" -> 3
7
8 cout << slq.lcpEntreSufijos(0, 3);
9 // sufijo desde 0: "banana"
10 // sufijo desde 3: "ana"
11 // LCP = "" -> 0

```

Nota: buildSuffixArray y buildLCPArray son las funciones estándar de Suffix Array + Kasai (ver la sección de Algoritmos de Strings); aquí solo se reutilizan como precómputo antes de aplicar RMQ.

Regla práctica: cuando un problema pida "algo" entre dos elementos que no son adyacentes en su estructura original (dos nodos de un árbol, dos sufijos de un string, dos elementos de cualquier secuencia con una noción de "orden" y "profundidad/afinidad"), pregúntate si esa relación se puede expresar como el mínimo de un arreglo auxiliar construido con un recorrido lineal. Si sí, el problema se reduce a Sparse Table (estático) o Segment Tree (dinámico), ya cubiertos en las secciones anteriores.

5.8 Mo's Algorithm

Idea: responde q queries de rango **offline** (las conoces todas de antemano) en $O((n + q)\sqrt{n})$ total, sin ninguna estructura de datos elaborada. La clave es **ordenar las queries** —no procesarlas en el orden dado—

agrupándolas por bloque de tamaño $\sqrt{n}$ según su extremo izquierdo l , y dentro de cada bloque por su extremo derecho r (alternando la dirección en bloques pares/impares para aprovechar mejor la localidad). Luego mantienes un puntero $[curL, curR]$ que se **desliza** de una query a la siguiente con funciones **add/remove** en $O(1)$ cada una. Como los punteros nunca retroceden más de lo necesario gracias al orden elegido, el total de movimientos amortizado es $O((n+q)\sqrt{n})$ en vez de $O(nq)$ con fuerza bruta.

¿Por qué usarlo? Sirve para respuestas que **no tienen una estructura de rango obvia** (por ejemplo: contar elementos distintos, sumar cuadrados de frecuencias, contar pares con cierta propiedad) donde escribir un Segment Tree específico sería complicado o imposible, pero sí es fácil mantener la respuesta incrementalmente al agregar/quitar un elemento a la vez.

Casos de uso:

- Contar elementos distintos en un rango, con muchas queries.
- Sumas o conteos que dependen de *frecuencias* dentro del rango (no solo de valores individuales), donde **add/remove** son fáciles pero no hay forma clara de combinar dos sub-rangos con un Segment Tree.
- Todas las queries se conocen de antemano (offline). Si necesitas responder **online** (una query a la vez, sin ver las siguientes), Mo's no aplica: usa Segment Tree o Sqrt Decomposition.
- El arreglo tiene pocos updates puntuales intercalados con las queries: usa la variante *Mo's con updates* (3D Mo's).

Complejidad: $O((n+q)\sqrt{n})$ total para q queries sin updates; $O(n^{5/3})$ (aprox.) para la variante con updates, usando bloques de tamaño $n^{2/3}$.

Mo's clásico (queries offline, sin updates)

Ejemplo guía: contar cuántos valores **distintos** hay en cada rango $[l, r]$ preguntado.

```
1 struct MoAlgorithm {
2     int n, blockSize;
3     vector<int> a;
4     vector<long long> cnt;
5     int curL = 0, curR = -1;
6     long long curDistinct = 0;
7
8     // CREATE: reserva el arreglo de frecuencias segun el maximo
9     // valor posible, y fija el tamaño de bloque en  $\sqrt{n}$ .
10    MoAlgorithm(vector<int>& arr, int maxVal) {
11        a = arr;
12        n = a.size();
13        blockSize = max(1, (int)sqrt((double)n));
14        cnt.assign(maxVal, 0);
15    }
16
17    // UPDATE: agrega el elemento en pos a la ventana actual.
18    void add(int pos) {
19        if (cnt[a[pos]]++ == 0)
20            curDistinct++;
21    }
22
23    // UPDATE: quita el elemento en pos de la ventana actual.
24    void remove(int pos) {
25        if (--cnt[a[pos]] == 0)
26            curDistinct--;
27    }
28
29    // READ: procesa todas las queries offline (l, r inclusive,
```

```
30    // 0-indexadas) y devuelve las respuestas en el orden
31    // original.
32    vector<long long> resolver(vector<array<int, 2>>& queries) {
33        int q = queries.size();
34        vector<int> idx(q);
35        iota(idx.begin(), idx.end(), 0);
36
37        sort(idx.begin(), idx.end(), [&](int i, int j) {
38            int bi = queries[i][0] / blockSize;
39            int bj = queries[j][0] / blockSize;
40            if (bi != bj) return bi < bj;
41            return (bi & 1) ? (queries[i][1] > queries[j][1])
42                           : (queries[i][1] < queries[j][1]);
43        });
44
45        vector<long long> respuestas(q);
46        for (int id : idx) {
47            int l = queries[id][0], r = queries[id][1];
48
49            while (curR < r) add(++curR);
50            while (curL > l) add(--curL);
51            while (curR > r) remove(curR--);
52            while (curL < l) remove(curL++);
53
54            respuestas[id] = curDistinct;
55        }
56        return respuestas;
57    }
58    // O((n+q) sqrt(n))
```

Ejemplo de funcionamiento:

```
1 vector<int> a = {1, 2, 1, 3, 2, 4, 1};
2 // valores en [1, 4]
3
4 MoAlgorithm mo(a, 5);
5
6 vector<array<int, 2>> queries = {
7     {0, 2}, // [1, 2, 1] -> 2 distintos
8     {1, 4}, // [2, 1, 3, 2] -> 3 distintos
9     {0, 6} // arreglo completo -> 4 distintos
10 };
11
12 vector<long long> resp = mo.resolver(queries);
13 // resp = [2, 3, 4]
```

Mo's con updates (3D Mo's)

Idea: si además de queries hay **updates puntuales** intercaladas, se agrega una tercera dimensión al ordenamiento: el *tiempo* t (cuántas updates ya ocurrieron antes de cada query). Se ordenan las queries por (bloque(l), bloque(r), t), y además de deslizar $[curL, curR]$, se avanza o retrocede $curT$ aplicando o revertiendo updates una por una. El bloque óptimo cambia a $\approx n^{2/3}$ (en vez de $\sqrt{n}$) porque ahora hay tres dimensiones que balancear.

```
1 struct MoUpdates {
2     int n, blockSize;
3     vector<int> a;
4     vector<long long> cnt;
5     vector<pair<int, int>> updates; // (pos, valor); ver nota
6     // abajo
7     int curL = 0, curR = -1, curT = 0;
8     long long curDistinct = 0;
9
10    // CREATE: reserva frecuencias y fija el bloque en  $n^{2/3}$ .
11    MoUpdates(vector<int>& arr, int maxVal) {
12        a = arr;
13        n = a.size();
14        blockSize = max(1, (int)cbrt((double)n) *
15                        (int)cbrt((double)n));
16        cnt.assign(maxVal, 0);
17    }
18
19    // UPDATE: agrega el elemento en pos a la ventana actual.
20    void add(int pos) {
21        if (cnt[a[pos]]++ == 0) curDistinct++;
22    }
23
24    // UPDATE: quita el elemento en pos de la ventana actual.
25    void remove(int pos) {
26        if (--cnt[a[pos]] == 0) curDistinct--;
27    }
28
29    // CREATE: registra una update (aplicarla no es inmediato,
30    // solo queda en la lista para procesarse en su momento).
```

```

28 void agregarUpdate(int pos, int val) {
29     updates.push_back({pos, val});
30 } // O(1)
31
32 // UPDATE: aplica (o revierte, es la misma operacion) la
33 // update numero t. El truco: como updates[t].second guarda
34 // "el valor a poner", tras aplicarla queda guardado el valor
35 // viejo -> llamar de nuevo revierte exactamente al estado
36 // anterior.
37 void alternarUpdate(int t) {
38     int pos = updates[t].first;
39     int val = updates[t].second;
40
41     if (pos >= curL && pos <= curR)
42         remove(pos);
43
44     swap(a[pos], val);
45     updates[t].second = val;
46
47     if (pos >= curL && pos <= curR)
48         add(pos);
49 } // O(1)
50
51 // READ: procesa todas las queries offline (l, r, t) y
52 // devuelve las respuestas en el orden original.
53 vector<long long> resolver(vector<array<int, 3>>& queries) {
54     int q = queries.size();
55     vector<int> idx(q);
56     iota(idx.begin(), idx.end(), 0);
57
58     sort(idx.begin(), idx.end(), [&](int i, int j) {
59         int bi = queries[i][0] / blockSize;
60         int bj = queries[j][0] / blockSize;
61         if (bi != bj) return bi < bj;
62
63         int ri = queries[i][1] / blockSize;
64         int rj = queries[j][1] / blockSize;
65         if (ri != rj) return ri < rj;
66
67         return queries[i][2] < queries[j][2];
68     });
69
70     vector<long long> respuestas(q);
71     for (int id : idx) {
72         int l = queries[id][0], r = queries[id][1],
73             t = queries[id][2];
74
75         while (curR < r) add(++curR);
76         while (curL > l) add(--curL);
77         while (curR > r) remove(curR--);
78         while (curL < l) remove(curL++);
79
80         while (curT < t) alternarUpdate(curT++);
81         while (curT > t) alternarUpdate(--curT);
82
83         respuestas[id] = curDistinct;
84     }
85     return respuestas;
86 } // O(n^(5/3))
87 } // aprox.

```

Ejemplo de funcionamiento:

```

1 vector<int> a = {1, 2, 3};
2
3 MoUpdates mo(a, 5);
4 mo.agregarUpdate(0, 2); // update #0: a[0] = 2
5 mo.agregarUpdate(1, 1); // update #1: a[1] = 1
6
7 // queries: (l, r, cuantas updates ya se aplicaron)
8 vector<array<int, 3>> queries = {
9     {0, 2, 0}, // arreglo original [1,2,3] -> 3 distintos
10    {0, 2, 1}, // tras update #0: [2,2,3] -> 2 distintos
11    {0, 2, 2}  // tras update #1: [2,1,3] -> 3 distintos
12 };
13
14 vector<long long> resp = mo.resolver(queries);
15 // resp = [3, 2, 3]

```

Nota: para problemas sobre **árboles** (queries sobre un camino o subárbol) existe *Mo's on Trees*, que aplanar el árbol con un Euler Tour y aplica el mismo algoritmo sobre ese arreglo; el resto del razonamiento (ordenar queries, deslizar punteros) es idéntico.

Comparación con otras estructuras de rango:

Situación	Estructura	Complejidad
Queries offline, sin updates	Mo's clásico	$O((n + q)\sqrt{n})$
Queries offline, con updates	Mo's con updates	$O(n^{5/3})$
Queries online (una a la vez)	Segment Tree / Sqrt Decomp.	$O(\log n)$ / $O(\sqrt{n})$

Table 8: Cuándo usar Mo's Algorithm frente a estructuras online.

Regla práctica: usa Mo's solo cuando (1) todas las queries son offline, y (2) la respuesta es fácil de mantener incrementalmente con **add/remove**, pero difícil de combinar con un Segment Tree (min/max/suma si se combinan fácil, así que ahí Mo's no aporta nada). Si el problema admite Segment Tree o Fenwick directamente, esas opciones son más simples y no necesitan que las queries sean offline.

5.9 Wavelet Tree

Idea: es un árbol binario sobre el **rango de valores** del arreglo (no sobre las posiciones, como Segment Tree). La raíz cubre $[lo, hi]$ de valores posibles; en cada nodo, los elementos con valor $\leq$ mediana van al hijo izquierdo y el resto al derecho, **preservando su orden relativo** (partición estable). Cada nodo guarda un arreglo $b[]$ con cuántos elementos, hasta cada posición, fueron hacia la izquierda. Con eso puedes, en cada nivel, saber exactamente qué sub-rango de posiciones le corresponde a cada hijo — y bajar por el árbol respondiendo preguntas sobre **valores dentro de un rango de posiciones** en $O(\log C)$, donde C es la cantidad de valores distintos.

¿Por qué usarlo? Es la única estructura de esta sección que responde eficientemente preguntas que combinan **posición y valor** a la vez: "el k -ésimo menor en $[l, r]$ ", "cuántos elementos en $[l, r]$ son $\leq X$ ", "cuántas veces aparece exactamente X en $[l, r]$ ". Ninguna de las estructuras anteriores (Fenwick, Segment Tree, Sparse Table) resuelve esto directamente sin re-plantear el problema.

Casos de uso:

- k -ésimo elemento más chico (o más grande) dentro de un rango $[l, r]$.
- Conteo de elementos en $[l, r]$ cuyo valor está en un rango $[a, b]$.
- Frecuencia de un valor específico dentro de un rango de posiciones (rank query).
- Arreglo **estático** — no soporta updates sin reconstruir (para eso existe la variante persistente, más avanzada).

Complejidad: construir $O(n \log C)$, cada query $O(\log C)$, donde C es la cantidad de valores distintos tras comprimir coordenadas.

Wavelet Tree: k -ésimo, conteo $\leq X$ y frecuencia

```

1 struct WaveletTree {
2     int lo, hi;
3     WaveletTree *izq, *der = nullptr;
4     vector<int> b; // b[i] = cuantos de a[0..i-1] fueron a la
5     izq.

```



```

6 // CREATE: construye recursivamente sobre el segmento [from,
7 // de un arreglo mutable, particionando por la mediana de
8 // [lo, hi]. Los valores deben venir ya comprimidos a [0, C
9 WaveletTree(int* from, int* to, int lo, int hi) : lo(lo), hi(hi) {
10     if (lo == hi || from == to) return;
11
12     int mid = (lo + hi) / 2;
13     auto valzq = [mid](int x) { return x <= mid; };
14
15     b.reserve(to - from + 1);
16     b.push_back(0);
17     for (auto it = from; it != to; it++)
18         b.push_back(b.back() + valzq(*it));
19
20     auto pivote = stable_partition(from, to, valzq);
21     izq = new WaveletTree(from, pivote, lo, mid);
22     der = new WaveletTree(pivote, to, mid + 1, hi);
23 } // O(n log C)
24
25 // READ: k-esimo valor mas chico en [l, r] (1-indexado, k
26 // desde 1).
27 int kth(int l, int r, int k) {
28     if (l > r) return -1;
29     if (lo == hi) return lo;
30
31     int enIzq = b[r] - b[l - 1];
32     int lb = b[l - 1], rb = b[r];
33
34     if (k <= enIzq)
35         return izq->kth(lb + 1, rb, k);
36     return der->kth(l - lb, r - rb, k - enIzq);
37 } // O(log C)
38
39 // READ: cuantos elementos de [l, r] tienen valor <= val.
40 int contarMenorIgual(int l, int r, int val) {
41     if (l > r || val < lo) return 0;
42     if (hi <= val) return r - l + 1;
43
44     int lb = b[l - 1], rb = b[r];
45     return izq->contarMenorIgual(lb + 1, rb, val)
46         + der->contarMenorIgual(l - lb, r - rb, val);
47 } // O(log C)
48
49 // READ: cuantas veces aparece exactamente val en [l, r].
50 int contarIgual(int l, int r, int val) {
51     if (l > r || val < lo || val > hi) return 0;
52     if (lo == hi) return r - l + 1;
53
54     int mid = (lo + hi) / 2;
55     int lb = b[l - 1], rb = b[r];
56
57     if (val <= mid)
58         return izq->contarIgual(lb + 1, rb, val);
59     return der->contarIgual(l - lb, r - rb, val);
60 } // O(log C)
61
62 ~WaveletTree() {
63     delete izq;
64     delete der;
65 }

```

Ejemplo de funcionamiento (con compresión de coordenadas):

```

1 vector<int> original = {40, 10, 30, 10, 20, 40, 30};
2
3 // comprimir a [0, C-1] para que el wavelet tree trabaje con
4 // valores pequenos y contiguos.
5 vector<int> ordenado = original;
6 sort(ordenado.begin(), ordenado.end());
7 ordenado.erase(unique(ordenado.begin(), ordenado.end()), ordenado
8 .end());
9
10 vector<int> comprimido(original.size());
11 for (int i = 0; i < (int)original.size(); i++)
12     comprimido[i] = lower_bound(ordenado.begin(), ordenado.end(),
13         original[i]) - ordenado.begin();
14
15 // comprimido: [3, 0, 2, 0, 1, 3, 2] (valores 10,20,30,40 ->
16 // 0,1,2,3)
17
18 WaveletTree wt(comprimido.data(),
19     comprimido.data() + comprimido.size(),
20     0, (int)ordenado.size() - 1);
21
22 // 3er elemento mas chico en el rango [2, 6] (1-indexado)
23 int idx = wt.kth(2, 6, 3);
24 cout << ordenado[idx];

```

```

22 // posiciones 2..6 (1-idx): 10,30,10,20,40,30 (valores 0-idx 2..6)
23
24 // -> valores reales: 30,10,20,40,30 ordenados: 10,20,30,30,40
25 // 3er mas chico = 30
26
27 cout << wt.contarMenorIgual(1, 7, 1);
28 // cuantos valores comprimidos <= 1 (o sea, <= 20) hay en todo el
29 // arreglo
30 // original: 40,10,30,10,20,40,30 -> <=20 son: 10,10,20 -> 3
31
32 cout << wt.contarIgual(1, 7, 0);
33 // cuantas veces aparece el valor comprimido 0 (o sea, 10) -> 2

```

Comparación con las otras estructuras de rango:

Pregunta	Estructura
Suma/min/max de rango	Fenwick / Segment Tree / Sparse Table
k-ésimo menor, o conteo por rango de valores , en un rango de posiciones	Wavelet Tree
Elementos distintos, moda, funciones no lineales de frecuencia	Mo's Algorithm

Table 9: Cuándo el problema necesita Wavelet Tree en vez de las otras estructuras.

Regla práctica: si el enunciado combina una condición sobre **valores** (“menor que”, “entre a y b ”, “el k -ésimo más chico”) con un rango de **posiciones**, y el arreglo es estático, es casi siempre Wavelet Tree. Si solo te piden agregar (suma/min/max) sin condición sobre el valor, usa las estructuras más simples de las secciones anteriores.

5.10 Heavy-Light Decomposition

Idea: descompone un árbol en **cadenas** de forma que cualquier camino desde la raíz hasta cualquier nodo cruza a lo más $O(\log n)$ cadenas distintas. En cada nodo, el hijo con el **subárbol más grande** (el hijo “pesado”) continúa la misma cadena que su padre; los demás hijos inician cadenas nuevas. Al numerar los nodos en el orden en que se visitan (DFS que siempre sigue primero al hijo pesado), cada cadena queda como un **rango contiguo** de posiciones — así que una query sobre un **camino** del árbol se convierte en $O(\log n)$ queries de rango sobre un Segment Tree normal, una por cada cadena que el camino atraviesa.

¿Por qué usarlo? Es la forma estándar de llevar range queries de un arreglo a un **árbol**: sumas/mínimos/updates en el camino entre dos nodos, o en el subárbol de un nodo. Sin HLD, un camino arbitrario no tiene ninguna estructura contigua sobre la cual aplicar Fenwick o Segment Tree directamente.

Casos de uso:

- Suma/mínimo/máximo en el camino entre dos nodos u y v .
- Actualizar todos los nodos (o aristas) de un camino $u-v$.
- Consultas o updates sobre el **subárbol** completo de un nodo (más simple: el subárbol siempre es un rango contiguo, sin necesidad de descomponer en cadenas).
- Cualquier problema de árbol que “se sentiría” como Range Queries si el árbol fuera un arreglo lineal.

Complejidad: construir $O(n)$ (dos DFS) más $O(n)$ del Segment Tree subyacente; query o update de **camino** $O(\log^2 n)$ ($O(\log n)$ cadenas $\times$ $O(\log n)$ por consulta al Segment Tree); query o update de **subárbol** $O(\log n)$ (un solo rango contiguo).

HLD: suma en camino, suma en subárbol

Idea de la implementación: dos DFS iterativas (con pila explícita, para evitar stack overflow en árboles muy profundos o muy desbalanceados — un riesgo real con recursión cuando $n \geq 10^5$ y el árbol es una cadena larga). El primero (dfsTamano) calcula el tamaño de cada subárbol y decide el hijo pesado de cada nodo. El segundo (dfsDescomponer) recorre siempre primero al hijo pesado (así la cadena sigue consecutiva en las posiciones) y solo después explora los hijos livianos, cada uno iniciando su propia cadena nueva. Las posiciones resultantes (pos[]) alimentan un Segment Tree normal —aquí se reutiliza SegmentTreeLazyAdd de la sección anterior— sobre el cual se hacen las queries reales.

```
1 struct HLD {
2     vector<vector<int>> adj;
3     vector<int> padre, profundidad, tamano, pesado, cabeza, pos;
4     int n, timer = 0;
5     SegmentTreeLazyAdd* seg; // Segment Tree sobre el orden pos
6     // CREATE: dos DFS iterativas para calcular cadenas, luego
7     // arma
8     // el Segment Tree sobre el arreglo reordenado por pos[].
9     HLD(vector<vector<int>>& adjacency, vector<long long>&
10         valores,
11         int raiz) {
12         adj = adjacency;
13         n = adj.size();
14         padre.assign(n, -1);
15         profundidad.assign(n, 0);
16         tamano.assign(n, 1);
17         pesado.assign(n, -1);
18         cabeza.assign(n, raiz);
19         pos.assign(n, 0);
20
21         dfsTamano(raiz);
22         dfsDescomponer(raiz);
23
24         vector<long long> reordenado(n);
25         for (int v = 0; v < n; v++)
26             reordenado[pos[v]] = valores[v];
27
28         seg = new SegmentTreeLazyAdd(reordenado);
29     } // O(n)
30
31     // Fase 1: obtiene un orden donde cada padre aparece ANTES
32     // que
33     // sus hijos (equivalente a un preorden), usando una pila.
34     // Fase 2: recorre ese orden al revés (hijos antes que padres
35     // para acumular tamano[] y decidir el hijo pesado, igual que
36     // haría la versión recursiva al "volver" de cada llamada.
37     void dfsTamano(int raiz) {
38         vector<int> orden;
39         stack<int> st;
40         st.push(raiz);
41         padre[raiz] = -1;
42         profundidad[raiz] = 0;
43
44         while (!st.empty()) {
45             int u = st.top();
46             st.pop();
47             orden.push_back(u);
48
49             for (int v : adj[u]) {
50                 if (v != padre[u]) {
51                     padre[v] = u;
52                     profundidad[v] = profundidad[u] + 1;
53                     st.push(v);
54                 }
55             }
56         }
57
58         for (int i = (int)orden.size() - 1; i >= 0; i--) {
59             int u = orden[i];
```

```
60         tamano[u] = 1;
61         int maxSub = 0;
62
63         for (int v : adj[u]) {
64             if (v == padre[u]) continue;
65
66             tamano[u] += tamano[v];
67             if (tamano[v] > maxSub) {
68                 maxSub = tamano[v];
69                 pesado[u] = v;
70             }
71         }
72     } // O(n)
73
74     // Pila de pares (nodo, cabeza de su cadena). Para que el
75     // hijo
76     // pesado quede en el tope de la pila (y por lo tanto se
77     // procese inmediatamente despues, como en la versión
78     // recursiva), se empuja de ULTIMO: los hijos livianos
79     // (cadena nueva) se empujan primero.
80     void dfsDescomponer(int raiz) {
81         stack<pair<int, int>> st;
82         st.push({raiz, raiz});
83
84         while (!st.empty()) {
85             auto [u, h] = st.top();
86             st.pop();
87
88             cabeza[u] = h;
89             pos[u] = timer++;
90
91             for (int v : adj[u]) {
92                 if (v == padre[u] || v == pesado[u]) continue;
93                 st.push({v, v}); // cadena nueva
94             }
95
96             if (pesado[u] != -1)
97                 st.push({pesado[u], h}); // misma cadena
98         } // O(n)
99
100     // READ: suma de todos los nodos en el camino u-v.
101     long long consultarCamino(int u, int v) {
102         long long resultado = 0;
103
104         while (cabeza[u] != cabeza[v]) {
105             if (profundidad[cabeza[u]] < profundidad[cabeza[v]])
106                 swap(u, v);
107
108             resultado += seg->rango(pos[cabeza[u]], pos[u]);
109             u = padre[cabeza[u]];
110         }
111
112         if (profundidad[u] > profundidad[v]) swap(u, v);
113         resultado += seg->rango(pos[u], pos[v]);
114         return resultado;
115     } // O(log^2 n)
116
117     // UPDATE: suma val a todos los nodos en el camino u-v.
118     void actualizarCamino(int u, int v, long long val) {
119         while (cabeza[u] != cabeza[v]) {
120             if (profundidad[cabeza[u]] < profundidad[cabeza[v]])
121                 swap(u, v);
122
123             seg->rangoUpdate(pos[cabeza[u]], pos[u], val);
124             u = padre[cabeza[u]];
125         }
126
127         if (profundidad[u] > profundidad[v]) swap(u, v);
128         seg->rangoUpdate(pos[u], pos[v], val);
129     } // O(log^2 n)
130
131     // READ: suma de todo el subarbol de u (rango contiguo por
132     // construcción, gracias al orden DFS de dfsDescomponer).
133     long long consultarSubarbol(int u) {
134         return seg->rango(pos[u], pos[u] + tamano[u] - 1);
135     } // O(log n)
136
137     // UPDATE: suma val a todo el subarbol de u.
138     void actualizarSubarbol(int u, long long val) {
139         seg->rangoUpdate(pos[u], pos[u] + tamano[u] - 1, val);
140     } // O(log n)
141 };
```

Ejemplo de funcionamiento:

```
1 // arbol:      0
2 //             /\
3 //            1 2 3
4 //             /\
5 //            4 5
```



```

7 vector<vector<int>> adj(6);
8 adj[0] = {1, 2, 3};
9 adj[1] = {0, 4, 5};
10 adj[2] = {0};
11 adj[3] = {0};
12 adj[4] = {1};
13 adj[5] = {1};
14
15 vector<long long> valores = {1, 2, 3, 4, 5, 6};
16 // nodo:      0 1 2 3 4 5
17
18 HLD hld(adj, valores, 0);
19
20 cout << hld.consultarCamino(4, 3);
21 // camino 4 -> 1 -> 0 -> 3 : valores 5+2+1+4 = 12
22
23 cout << hld.consultarSubarbol(1);
24 // subarbol de 1: nodos {1,4,5} : 2+5+6 = 13
25
26 hld.actualizarCamino(4, 5, 10);
27 // suma 10 a cada nodo del camino 4 -> 1 -> 5
28
29 cout << hld.consultarSubarbol(0);
30 // subarbol de 0 (todo el arbol) tras el update:
31 // 1+(2+10)+3+4+(5+10)+(6+10) = 51

```

Nota: al ser iterativas, `dfsTamano` y `dfsDescomponer` no tienen riesgo de stack overflow incluso en árboles con $n \geq 10^5$ y forma de “cadena larga” (el peor caso para recursión). `SegmentTreeLazyAdd` es exactamente el struct de la sección de Segment Tree — cualquier otra variante (min, max, assign) funciona igual de bien como motor subyacente.

Regla práctica: usa HLD en cuanto el problema pida algo sobre un **camino** entre dos nodos de un árbol con muchas queries. Si solo pide algo sobre **subárboles** (sin caminos arbitrarios), no hace falta la descomposición en cadenas completa: basta un solo DFS para las posiciones (`pos[]` y `tamano[]`) y un Segment Tree normal, ya que el subárbol siempre es un rango contiguo por sí solo.

Parte III Grafos

5.11 ¿Qué algoritmo usar?

La siguiente tabla resume situaciones típicas de Competitive Programming y el algoritmo que debe considerarse. La idea es reconocer las palabras clave o condiciones del enunciado y asociarlas rápidamente con una técnica.

Cuando el ejercicio diga / pida	Usa	Pista rápida
“Recorrer el grafo por niveles”	BFS	Usa una cola.
“Encontrar la distancia mínima” con aristas de igual costo	BFS	Cada arista cuesta lo mismo.
“Encontrar caminos mínimos” con pesos 0 y 1	0-1 BFS	Deque; pesos únicamente 0 o 1.
“Camino mínimo” con pesos no negativos	Dijkstra	Usa una cola de prioridad.
“Camino mínimo” con pesos negativos	Bellman-Ford	Permite detectar ciclos negativos.
“Camino mínimo entre todos los pares” y n pequeño	Floyd-Warshall	DP sobre todos los pares.
“¿Existe un camino entre u y v ?”	DFS / BFS	Problema de conectividad.
“Encontrar componentes conexas”	DFS / BFS	Ejecutar traversal desde cada nodo no visitado.
“Detectar si existe un ciclo” en grafo no dirigido	DFS / DSU	DFS para recorrer; DSU si se agregan aristas.
“Detectar ciclos” en grafo dirigido	DFS / Kahn	Usar estados en DFS o grados de entrada.
“Ordenar tareas con dependencias”	Topological Sort	El grafo debe ser dirigido y acíclico.
“Encontrar puentes”	Tarjan	DFS + tiempos de descubrimiento.
“Encontrar puntos de articulación”	Tarjan	Vértices cuya eliminación desconecta el grafo.
“Encontrar componentes fuertemente conexas”	Kosaraju / Tarjan	Grafo dirigido.
“Conectar todos los nodos con costo mínimo”	MST	Kruskal o Prim.
“MST procesando aristas ordenadas por peso”	Kruskal	Ordenar aristas + DSU.
“MST expandiendo desde un nodo”	Prim	Priority Queue + lista de adyacencia.
“¿Dos nodos pertenecen al mismo conjunto?”	DSU	Union-Find.
“Agrupar elementos mediante uniones”	DSU	union + find.
“Encontrar si un grafo es bipartito”	2-Coloring	Colorear con dos colores.
“Máximo número de parejas entre dos grupos”	Bipartite Matching	Emparejamiento entre conjuntos.
“Máximo flujo de una fuente a un destino”	Dinic	Network Flow.
“Separar fuente y sumidero con capacidad mínima”	Min-Cut	Por el teorema Max-Flow Min-Cut.
“Distancia entre dos nodos de un árbol”	LCA	$\text{dist}(u, v) = d[u] + d[v] - 2d[\text{LCA}(u, v)]$.
“Encontrar el ancestro k niveles arriba”	Binary Lifting	Saltar potencias de 2.
“Consultas sobre subárboles”	Euler Tour + Fenwick / Segment Tree	Convertir subárbol en intervalo.
“Consultas o actualizaciones sobre caminos de un árbol”	HLD	Descomponer el camino en cadenas.
“Encontrar el diámetro de un árbol”	BFS / DFS	Dos traversals desde extremos.
“Encontrar un camino que use todas las aristas”	Eulerian Path / Hierholzer	Trabaja con aristas, no con vértices.
“Encontrar todas las rutas posibles”	DFS + Backtracking	Explorar y deshacer decisiones.
“Rellenar una región conectada”	Flood Fill	DFS / BFS desde una celda inicial.
“Moverse por una cuadrícula buscando la menor cantidad de pasos”	BFS	Cada movimiento tiene costo 1.
“Variables booleanas con restricciones de la forma OR”	2-SAT + SCC	Construir grafo de implicaciones.
“Resolver muchas consultas sobre un árbol”	LCA / HLD	Depende de si son ancestros o caminos.

Table 10: Guía rápida para seleccionar algoritmos de grafos.

Condición del problema	Algoritmo	Complejidad
Aristas sin peso / costo 1	BFS	$O(n + m)$
Pesos únicamente 0 y 1	0-1 BFS	$O(n + m)$
Pesos no negativos	Dijkstra	$O((n + m) \log n)$
Puede haber pesos negativos	Bellman-Ford	$O(nm)$
Todos los pares y n pequeño	Floyd-Warshall	$O(n^3)$

Table 11: Selección del algoritmo para caminos mínimos.

6 Traversal

6.1 Representación de Grafos

Idea: antes de aplicar cualquier algoritmo sobre un grafo, primero debemos decidir cómo almacenar sus vértices y aristas. Las tres representaciones más comunes son la **lista de adyacencia**, la **matriz de adyacencia** y la **lista de aristas**. La elección depende principalmente de la cantidad de vértices y aristas y del tipo de consultas que realizará el algoritmo.

La **lista de adyacencia** almacena, para cada vértice, todos los vértices conectados directamente con él. Es la representación más utilizada en Competitive Programming porque ocupa $O(n + m)$ memoria y permite recorrer eficientemente todos los vecinos de un vértice.

La **matriz de adyacencia** utiliza una matriz $n \times n$ donde la posición (u, v) indica si existe una arista entre u y v . Permite comprobar la existencia de una arista en $O(1)$, pero requiere $O(n^2)$ memoria.

La **lista de aristas** simplemente almacena cada arista como un par (u, v) , o como una terna (u, v, w) si tiene peso. Es especialmente útil para algoritmos que necesitan procesar todas las aristas, como Kruskal o Bellman-Ford.

¿Por qué usar cada una?

- **Lista de adyacencia:** opción general para BFS, DFS, Dijkstra, Prim, búsqueda de ciclos y prácticamente la mayoría de algoritmos sobre grafos dispersos.
- **Matriz de adyacencia:** útil cuando n es pequeño o cuando se necesita comprobar constantemente si existe una arista entre dos vértices en $O(1)$.
- **Lista de aristas:** conveniente cuando el algoritmo necesita recorrer todas las aristas directamente, especialmente en Kruskal y Bellman-Ford.

Tipos de grafos: una arista puede ser **dirigida** o **no dirigida**, y puede tener o no un **peso**. En un grafo no dirigido, una arista (u, v) se representa agregando v a los vecinos de u y u a los vecinos de v . En un grafo dirigido, solamente se agrega v a los vecinos de u .

Complejidad:

Representación	Memoria	Consulta	Vecinos
Lista de adyacencia	$O(n + m)$	$O(\deg(u))$	$O(\deg(u))$
Matriz de adyacencia	$O(n^2)$	$O(1)$	$O(n)$
Lista de aristas	$O(m)$	$O(m)$	$O(m)$

Table 12: Comparación de representaciones de grafos.

Lista de adyacencia

Idea de la implementación: se utiliza un `vector<vector<int>>` donde la posición u contiene todos los vértices conectados directamente con u . Para grafos ponderados se puede almacenar un par (v, w) , donde v es el destino y w el peso de la arista.

```

1 // Grafo no dirigido y no ponderado.
2 int n = 5;
3 vector<vector<int>> adj(n);
4
5 // Agrega una arista u-v.
6 auto agregarArista = [&](int u, int v) {
7     adj[u].push_back(v);
8     adj[v].push_back(u);
9 };
10
11 agregarArista(0, 1);
12 agregarArista(0, 2);
13 agregarArista(1, 3);
14 agregarArista(1, 4);

```

Grafo resultante:

```

1 //      0
2 //      / \
3 //      1  2
4 //      / \
5 //      3  4
6 //
7 // adj[0] = {1, 2}
8 // adj[1] = {0, 3, 4}
9 // adj[2] = {0}
10 // adj[3] = {1}
11 // adj[4] = {1}

```

Para un grafo dirigido, solamente se agrega la arista en una dirección:

```

1 // Grafo dirigido.
2 vector<vector<int>> adj(n);
3
4 auto agregarArista = [&](int u, int v) {
5     adj[u].push_back(v);
6 };
7
8 agregarArista(0, 1);
9 agregarArista(0, 2);
10 agregarArista(1, 3);

```

Para un grafo ponderado, cada vecino puede almacenar también el peso:

```

1 // Grafo ponderado.
2 vector<vector<pair<int, long long>>> adj(n);
3
4 auto agregarArista = [&](int u, int v, long long w) {
5     adj[u].push_back({v, w});
6     adj[v].push_back({u, w});
7 };
8
9 agregarArista(0, 1, 5);
10 agregarArista(0, 2, 3);
11 agregarArista(1, 3, 7);

```

Ejemplo de recorrido:

```

1 void dfs(int u, int padre, vector<vector<int>>& adj) {
2
3     cout << u << ' ';
4
5     for (int v : adj[u]) {
6         if (v == padre) continue;
7
8         dfs(v, u, adj);
9     }
10 }

```

La lista de adyacencia permite que DFS o BFS procesen cada vértice y cada arista una cantidad constante de veces, obteniendo una complejidad total de $O(n + m)$.

Matriz de adyacencia

Idea: una matriz $adj[u][v]$ indica directamente si existe una conexión desde u hacia v . Si el grafo es ponderado, la posición puede almacenar directamente el peso de la arista.

```
1 int n = 5;
2 vector<vector<int>> adj(n, vector<int>(n, 0));
3
4 auto agregarArista = [&](int u, int v) {
5     adj[u][v] = 1;
6     adj[v][u] = 1;
7 };
8
9 agregarArista(0, 1);
10 agregarArista(0, 2);
11 agregarArista(1, 3);
12 agregarArista(1, 4);
```

Ahora comprobar si existe una arista entre u y v es inmediato:

```
1 if (adj[u][v]) {
2     cout << "Existe una arista";
3 }
```

La consulta anterior cuesta $O(1)$, pero recorrer todos los vecinos de u requiere revisar las n posiciones de la fila:

```
1 for (int v = 0; v < n; v++) {
2     if (adj[u][v]) {
3         cout << v << ' ';
4     }
5 }
```

Lista de aristas

Idea: almacena directamente todas las aristas del grafo. Para un grafo no ponderado basta guardar (u, v) ; para uno ponderado se guarda (u, v, w) .

```
1 // Grafo ponderado.
2 struct Edge {
3     int u, v;
4     long long w;
5 };
6
7 vector<Edge> edges;
8
9 edges.push_back({0, 1, 5});
10 edges.push_back({0, 2, 3});
11 edges.push_back({1, 3, 7});
```

Esta representación resulta especialmente conveniente cuando el algoritmo necesita procesar todas las aristas:

```
1 for (Edge e : edges) {
2     cout << e.u << " -> " << e.v
3         << " peso = " << e.w << '\n';
4 }
```

Por ejemplo, **Kruskal** ordena directamente esta lista por peso, mientras que **Bellman-Ford** puede recorrerla completa en cada iteración.

Nota: para la mayoría de problemas de Competitive Programming, la **lista de adyacencia** debe ser la representación por defecto. Es especialmente importante cuando el grafo es disperso, es decir, cuando m es mucho menor que n^2 . La matriz de adyacencia solamente conviene cuando n es pequeño o cuando las consultas de existencia de aristas en $O(1)$ son importantes.

Regla práctica: si no tienes una razón específica para usar otra representación, usa **lista de adyacencia**. Si necesitas preguntar constantemente “¿existe una arista $u \rightarrow v$?”, considera una **matriz de adyacencia**. Si

el algoritmo trabaja directamente con todas las aristas, considera una **lista de aristas**.

“latex

6.2 BFS (Breadth-First Search)

Idea: BFS (*Breadth-First Search*) es un algoritmo de recorrido de grafos que explora los vértices por **niveles**. A partir de un vértice inicial, primero visita todos sus vecinos directos, después los vecinos de estos, y continúa de esta manera hasta procesar todos los vértices alcanzables.

Para mantener este orden se utiliza una **cola** (*queue*), siguiendo el principio FIFO (*First In, First Out*). Cada vértice se marca como visitado cuando se agrega a la cola, evitando procesarlo varias veces.

Una de las propiedades más importantes de BFS es que, en un grafo **no ponderado**, encuentra la distancia mínima desde un vértice origen hasta cualquier otro vértice alcanzable, donde la distancia se mide como el número de aristas recorridas.

¿Por qué usarlo?

- **Camino mínimo:** encuentra el camino con menor número de aristas en grafos no ponderados.
- **Distancias:** permite calcular la distancia mínima desde un origen hasta todos los vértices alcanzables.
- **Recorrido por niveles:** permite procesar los vértices según su distancia al origen.
- **Reconstrucción de caminos:** almacenando el padre de cada vértice se puede recuperar el camino mínimo.
- **Bipartición:** permite determinar si un grafo es bipartito utilizando dos colores.
- **Componentes conexas:** ejecutando BFS desde cada vértice no visitado se pueden encontrar todas las componentes conexas.
- **Problemas en grids:** permite encontrar la cantidad mínima de movimientos cuando cada movimiento tiene el mismo costo.
- **Multi-Source BFS:** permite calcular la distancia mínima desde cualquiera de varios vértices iniciales.

Funcionamiento:

El algoritmo sigue los siguientes pasos:

1. Elegir un vértice inicial s .
2. Marcar s como visitado y agregarlo a la cola.
3. Mientras la cola no esté vacía, extraer el primer vértice.
4. Recorrer todos los vecinos del vértice extraído.
5. Si un vecino no ha sido visitado, marcarlo y agregarlo a la cola.

La cola garantiza que los vértices se procesen por niveles. Primero se procesan los vértices a distancia 0, después los de distancia 1, luego los de distancia 2, y así sucesivamente.

Implementación básica:

```
1 void bfs(int inicio, vector<vector<int>>& adj) {
2
3     int n = adj.size();
4
5     vector<bool> visitado(n, false);
6     queue<int> q;
7
8     visitado[inicio] = true;
9     q.push(inicio);
10
11     while (!q.empty()) {
12
13         int u = q.front();
14         q.pop();
15
16         cout << u << ' ';
17
18         for (int v : adj[u]) {
19
20             if (visitado[v]) continue;
21
22             visitado[v] = true;
23             q.push(v);
24         }
25     }
26 }
```

Ejemplo:

Consideremos el siguiente grafo:

```
1 //      0
2 //      / \
3 //     1   2
4 //    / \
5 //   3   4
```

Si comenzamos BFS desde el vértice 0, los niveles son:

```
1 // Nivel 0: 0
2 // Nivel 1: 1 2
3 // Nivel 2: 3 4
```

Por lo tanto, un posible orden de recorrido es:

```
1 0 1 2 3 4
```

El orden entre vértices del mismo nivel depende del orden en que aparezcan en la lista de adyacencia.

BFS para encontrar distancias:

Para calcular la distancia mínima desde el origen hasta cada vértice, se utiliza un arreglo `dist`. Inicialmente todos los valores son `-1`, indicando que el vértice todavía no ha sido visitado.

Cuando BFS descubre un nuevo vértice v desde u , su distancia se calcula como:

$$dist[v] = dist[u] + 1$$

```
1 vector<int> bfsDist(int inicio, vector<vector<int>>& adj) {
2
3     int n = adj.size();
4
5     vector<int> dist(n, -1);
6     queue<int> q;
7
8     dist[inicio] = 0;
9     q.push(inicio);
10
11     while (!q.empty()) {
12
13         int u = q.front();
14         q.pop();
15
16         for (int v : adj[u]) {
17
18             if (dist[v] != -1) continue;
19
20             dist[v] = dist[u] + 1;
```

```
21         q.push(v);
22     }
23 }
24
25 return dist;
26 }
```

Para el ejemplo anterior, comenzando desde 0, se obtiene:

```
1 // dist[0] = 0
2 // dist[1] = 1
3 // dist[2] = 1
4 // dist[3] = 2
5 // dist[4] = 2
```

Por ejemplo, $dist[3] = 2$ significa que el camino mínimo desde 0 hasta 3 utiliza dos aristas.

Reconstrucción del camino:

Si además de la distancia se necesita conocer el camino utilizado, se puede almacenar el **padre** de cada vértice. Cuando BFS descubre v desde u , se establece:

$$padre[v] = u$$

```
1 vector<int> dist(n, -1);
2 vector<int> padre(n, -1);
3
4 queue<int> q;
5
6 dist[inicio] = 0;
7 q.push(inicio);
8
9 while (!q.empty()) {
10
11     int u = q.front();
12     q.pop();
13
14     for (int v : adj[u]) {
15
16         if (dist[v] != -1) continue;
17
18         dist[v] = dist[u] + 1;
19         padre[v] = u;
20
21         q.push(v);
22     }
23 }
```

Una vez terminado BFS, el camino hasta un destino se reconstruye siguiendo los padres desde el destino hasta el origen:

```
1 vector<int> camino;
2
3 for (int v = destino; v != -1; v = padre[v]) {
4     camino.push_back(v);
5 }
6
7 reverse(camino.begin(), camino.end());
```

Por ejemplo, si:

```
1 // padre[3] = 1
2 // padre[1] = 0
3 // padre[0] = -1
```

el camino mínimo desde 0 hasta 3 es:

```
1 0 -> 1 -> 3
```

BFS en grafos no conectados:

Un BFS iniciado desde un único vértice solamente visita los vértices que pueden alcanzarse desde él. Para recorrer todo el grafo, se puede ejecutar BFS desde cada vértice que todavía no haya sido visitado.

```
1 vector<bool> visitado(n, false);
2
3 for (int s = 0; s < n; s++) {
4
5     if (visitado[s]) continue;
```

```

7   queue<int> q;
8
9   visitado[s] = true;
10  q.push(s);
11
12  while (!q.empty()) {
13
14      int u = q.front();
15      q.pop();
16
17      for (int v : adj[u]) {
18
19          if (visitado[v]) continue;
20
21          visitado[v] = true;
22          q.push(v);
23      }
24  }
25 }

```

Esta técnica permite encontrar, entre otras cosas, el número de **componentes conexas** de un grafo.

BFS para comprobar si un grafo es bipartito:

Un grafo es bipartito si sus vértices pueden dividirse en dos grupos de manera que ninguna arista conecte dos vértices del mismo grupo. BFS puede comprobar esta propiedad asignando alternativamente los colores 0 y 1 a los vértices.

Si un vértice u tiene color c , sus vecinos deben tener el color c^1 . Si una arista conecta dos vértices del mismo color, el grafo no es bipartito.

```

1 bool esBipartito(vector<vector<int>>& adj) {
2
3     int n = adj.size();
4
5     vector<int> color(n, -1);
6
7     for (int inicio = 0; inicio < n; inicio++) {
8
9         if (color[inicio] != -1) continue;
10
11         queue<int> q;
12
13         color[inicio] = 0;
14         q.push(inicio);
15
16         while (!q.empty()) {
17
18             int u = q.front();
19             q.pop();
20
21             for (int v : adj[u]) {
22
23                 if (color[v] == -1) {
24
25                     color[v] = color[u] ^ 1;
26                     q.push(v);
27
28                 } else if (color[v] == color[u]) {
29
30                     return false;
31                 }
32             }
33         }
34     }
35
36     return true;
37 }

```

El arreglo `color` también funciona como arreglo de visitados: un valor de -1 significa que el vértice todavía no ha sido procesado.

BFS en un grid:

BFS también puede utilizarse sobre matrices o tableros. Cada celda se considera un vértice y cada movimiento válido entre celdas se considera una arista.

Por ejemplo, si solamente se permiten movimientos hacia arriba, abajo, izquierda y derecha:

```

1 int dx[] = {-1, 1, 0, 0};
2 int dy[] = {0, 0, -1, 1};
3
4 queue<pair<int, int>> q;

```

```

5 q.push({sx, sy});
6 dist[sx][sy] = 0;
7
8 while (!q.empty()) {
9
10     auto [x, y] = q.front();
11     q.pop();
12
13     for (int d = 0; d < 4; d++) {
14
15         int nx = x + dx[d];
16         int ny = y + dy[d];
17
18         if (nx < 0 || nx >= n ||
19             ny < 0 || ny >= m)
20             continue;
21
22         if (dist[nx][ny] != -1)
23             continue;
24
25         if (grid[nx][ny] == '#')
26             continue;
27
28         dist[nx][ny] = dist[x][y] + 1;
29         q.push({nx, ny});
30     }
31 }
32 }

```

Esta técnica es especialmente útil en problemas donde se pregunta por la menor cantidad de movimientos necesarios para llegar desde una celda hasta otra.

Multi-Source BFS:

En algunos problemas existen varios vértices iniciales al mismo tiempo. En lugar de ejecutar un BFS separado desde cada uno, todos los orígenes se introducen inicialmente en la misma cola con distancia 0.

```

1 queue<int> q;
2 vector<int> dist(n, -1);
3
4 for (int s : fuentes) {
5
6     dist[s] = 0;
7     q.push(s);
8 }
9
10 while (!q.empty()) {
11
12     int u = q.front();
13     q.pop();
14
15     for (int v : adj[u]) {
16
17         if (dist[v] != -1) continue;
18
19         dist[v] = dist[u] + 1;
20         q.push(v);
21     }
22 }

```

De esta manera, cada vértice obtiene la distancia mínima hasta **cualquiera** de las fuentes.

Complejidad:

Utilizando una lista de adyacencia, cada vértice se procesa una sola vez y cada arista se revisa una cantidad constante de veces.

Representación	Tiempo	Memoria adicional
Lista de adyacencia	$O(n + m)$	$O(n)$
Matriz de adyacencia	$O(n^2)$	$O(n)$

Table 13: Complejidad de BFS según la representación del grafo.

La complejidad $O(n + m)$ hace que BFS sea especialmente eficiente para grafos dispersos representados mediante listas de adyacencia.

Nota: BFS encuentra caminos mínimos cuando todas las aristas tienen el mismo costo. No debe utilizarse directamente para encontrar caminos mínimos en grafos

con pesos arbitrarios. Para pesos no negativos se puede utilizar **Dijkstra**; si los pesos solamente son 0 y 1, una alternativa eficiente es **0-1 BFS**.

Regla práctica: si un problema pide minimizar el número de **pasos, movimientos o aristas** y cada movimiento tiene el mismo costo, considera primero **BFS**. Si necesitas conocer la distancia, utiliza **dist**; si necesitas recuperar el camino, utiliza **padre**; y si existen múltiples puntos de origen, considera **Multi-Source BFS**. “

“`latex`

6.3 BFS en Grafos Desconectados

Idea: un BFS iniciado desde un único vértice solamente visita los vértices que son alcanzables desde él. Si el grafo tiene varias **componentes conexas**, es necesario iniciar un nuevo BFS desde cada vértice que todavía no haya sido visitado.

La idea consiste en recorrer todos los vértices y, cada vez que se encuentra uno que no ha sido visitado, iniciar un nuevo BFS desde él. Cada ejecución de BFS procesa una componente conexas completa.

Implementación:

```
1 void bfs(int inicio, vector<vector<int>>& adj,
2       vector<bool>& visitado) {
3
4     queue<int> q;
5
6     visitado[inicio] = true;
7     q.push(inicio);
8
9     while (!q.empty()) {
10
11         int u = q.front();
12         q.pop();
13
14         for (int v : adj[u]) {
15
16             if (visitado[v]) continue;
17
18             visitado[v] = true;
19             q.push(v);
20         }
21     }
22 }
```

Para recorrer todo el grafo:

```
1 vector<bool> visitado(n, false);
2
3 for (int u = 0; u < n; u++) {
4
5     if (visitado[u]) continue;
6
7     bfs(u, adj, visitado);
8 }
```

Cada vez que se ejecuta **bfs**, se procesa una componente conexas que todavía no había sido visitada.

Contar componentes conexas:

Una aplicación directa consiste en contar cuántas componentes conexas tiene el grafo. Basta incrementar un contador cada vez que se inicia un nuevo BFS.

```
1 int componentes = 0;
2
3 vector<bool> visitado(n, false);
4
5 for (int u = 0; u < n; u++) {
6
7     if (visitado[u]) continue;
8
9     componentes++;
10
11     queue<int> q;
12
13     visitado[u] = true;
14     q.push(u);
15 }
```

```
16 while (!q.empty()) {
17
18     int v = q.front();
19     q.pop();
20
21     for (int w : adj[v]) {
22
23         if (visitado[w]) continue;
24
25         visitado[w] = true;
26         q.push(w);
27     }
28 }
29 }
30
31 cout << componentes << '\n';
```

Ejemplo:

Consideremos el siguiente grafo:

```
1 // 0 --- 1      3 --- 4
2 //           |
3 //           2
4 //
5 // 5 --- 6
```

El grafo tiene tres componentes conexas:

```
1 // Componente 1: {0, 1, 2}
2 // Componente 2: {3, 4}
3 // Componente 3: {5, 6}
```

Si recorremos los vértices en orden, el primer BFS puede comenzar desde 0 y visitar {0, 1, 2}. Después, los vértices 1 y 2 ya están visitados, por lo que el siguiente BFS comienza desde 3 y visita {3, 4}. Finalmente, se inicia otro BFS desde 5 para visitar {5, 6}.

Complejidad:

Aunque se ejecuten varios BFS, cada vértice se visita una sola vez y cada arista se procesa una cantidad constante de veces. Por lo tanto, la complejidad total utilizando una lista de adyacencia sigue siendo:

$$O(n + m)$$

La memoria adicional utilizada por el arreglo de visitados y la cola es $O(n)$.

Operación	Complejidad
Recorrer todas las componentes	$O(n + m)$
Contar componentes conexas	$O(n + m)$
Memoria adicional	$O(n)$

Table 14: Complejidad de BFS en grafos desconectados.

Regla práctica: si el grafo puede estar desconectado y se necesita procesar **todos los vértices**, no basta con ejecutar BFS una sola vez. Recorre todos los vértices y comienza un nuevo BFS cada vez que encuentres uno que todavía no haya sido visitado. “

6.4 BFS para Todos los Caminos

Idea: BFS no solamente permite encontrar la distancia mínima desde un vértice origen hacia los demás vértices. También puede utilizarse para reconstruir el **camino mínimo** desde el origen hasta cualquier vértice alcanzable.

Para reconstruir un camino, además de almacenar la distancia, se guarda para cada vértice su **padre**, es decir, el vértice desde el cual fue descubierto por primera vez. Como BFS visita los vértices por niveles, el primer padre asignado a un vértice pertenece a un camino mínimo.

Implementación:

```

1 vector<int> dist(n, -1);
2 vector<int> padre(n, -1);
3
4 queue<int> q;
5
6 dist[inicio] = 0;
7 q.push(inicio);
8
9 while (!q.empty()) {
10
11     int u = q.front();
12     q.pop();
13
14     for (int v : adj[u]) {
15
16         if (dist[v] != -1) continue;
17
18         dist[v] = dist[u] + 1;
19         padre[v] = u;
20
21         q.push(v);
22     }
23 }

```

Después de ejecutar BFS, `dist[v]` contiene la distancia mínima desde `inicio` hasta `v`, mientras que `padre[v]` indica qué vértice se debe seguir para reconstruir el camino.

Reconstrucción del camino:

Para obtener el camino hasta un vértice destino, se comienza en `destino` y se siguen los padres hasta llegar al origen.

```

1 vector<int> camino;
2
3 for (int v = destino; v != -1; v = padre[v]) {
4     camino.push_back(v);
5 }
6
7 reverse(camino.begin(), camino.end());

```

El vector `camino` contiene ahora los vértices en el orden correcto, desde el origen hasta el destino.

Ejemplo:

Supongamos que BFS produce los siguientes padres:

```

1 // padre[0] = -1
2 // padre[1] = 0
3 // padre[2] = 0
4 // padre[3] = 1
5 // padre[4] = 3

```

Si el origen es 0 y queremos reconstruir el camino hasta 4, seguimos los padres:

$$4 \rightarrow 3 \rightarrow 1 \rightarrow 0$$

Después de invertirlo obtenemos:

$$0 \rightarrow 1 \rightarrow 3 \rightarrow 4$$

Por lo tanto, este es un camino mínimo desde 0 hasta 4.

Función para reconstruir el camino:

La reconstrucción puede encapsularse en una función:

```

1 vector<int> reconstruirCamino(int destino,
2                             vector<int>& padre) {
3
4     vector<int> camino;
5
6     for (int v = destino; v != -1; v = padre[v]) {
7         camino.push_back(v);
8     }
9
10    reverse(camino.begin(), camino.end());
11
12    return camino;
13 }

```

El BFS completo puede combinarse con esta función:

```

1 vector<int> dist(n, -1);
2 vector<int> padre(n, -1);
3
4 queue<int> q;
5
6 dist[inicio] = 0;
7 q.push(inicio);
8
9 while (!q.empty()) {
10
11     int u = q.front();
12     q.pop();
13
14     for (int v : adj[u]) {
15
16         if (dist[v] != -1) continue;
17
18         dist[v] = dist[u] + 1;
19         padre[v] = u;
20
21         q.push(v);
22     }
23 }
24
25 vector<int> camino =
26     reconstruirCamino(destino, padre);

```

¿Qué significa “todos los caminos”?

En este contexto, BFS obtiene un camino mínimo desde el origen hacia **cada vértice alcanzable**. No significa que encuentre todos los caminos posibles entre dos vértices, ya que la cantidad de caminos posibles puede ser exponencial.

Por ejemplo, después de ejecutar BFS desde `s`:

- `dist[v]` indica la distancia mínima de `s` a `v`.
- `padre[v]` permite reconstruir un camino mínimo hacia `v`.
- Si `dist[v]` es `-1`, `v` no es alcanzable desde `s`.

Camino inexistente:

Antes de reconstruir el camino conviene comprobar si el destino fue alcanzado:

```

1 if (dist[destino] == -1) {
2     cout << "No existe camino\n";
3 }
4 else {
5     vector<int> camino =
6         reconstruirCamino(destino, padre);
7
8     for (int v : camino) {
9         cout << v << ' ';
10    }
11 }

```

Complejidad:

La ejecución de BFS utilizando una lista de adyacencia cuesta $O(n + m)$. La reconstrucción de un camino cuesta $O(k)$, donde k es la cantidad de vértices que contiene el camino.

Operación	Complejidad
BFS	$O(n + m)$
Reconstruir un camino	$O(k)$
Memoria adicional	$O(n)$

Table 15: Complejidad de BFS para reconstrucción de caminos.

Nota: BFS encuentra caminos mínimos en grafos donde todas las aristas tienen el mismo costo. Si las aristas tienen pesos diferentes, deben utilizarse algoritmos como **Dijkstra**, **0-1 BFS** o **Bellman-Ford**, dependiendo de las características del problema.

Regla práctica: si el problema pide no solamente la distancia mínima sino también **qué vértices forman el camino**, guarda un arreglo **padre** durante el BFS. No es necesario almacenar todos los caminos: basta con guardar un padre por vértice.

6.5 DFS

Idea: Depth-First Search (DFS) explora un grafo siguiendo un camino lo más profundamente posible antes de regresar y explorar otras ramas. A diferencia de BFS, que utiliza una **cola**, DFS utiliza una **pila**.

DFS puede implementarse de forma recursiva o iterativa. En Competitive Programming, la versión iterativa es útil cuando el grafo puede tener muchos vértices y una implementación recursiva podría provocar un **stack overflow**.

Implementación iterativa:

Se utiliza un `stack<int>` para almacenar los vértices que deben ser procesados. Un vértice se marca como visitado cuando se introduce en la pila, evitando que pueda agregarse múltiples veces.

```
1 void dfs(int inicio, vector<vector<int>>& adj,
2         vector<bool>& visitado) {
3
4     stack<int> st;
5
6     visitado[inicio] = true;
7     st.push(inicio);
8
9     while (!st.empty()) {
10
11         int u = st.top();
12         st.pop();
13
14         cout << u << ' ';
15
16         for (int v : adj[u]) {
17
18             if (visitado[v]) continue;
19
20             visitado[v] = true;
21             st.push(v);
22         }
23     }
24 }
```

El orden exacto en que se visitan los vértices depende del orden de los vecinos en la lista de adyacencia. Lo importante es que cada vértice alcanzable sea procesado una vez.

Ejemplo:

Consideremos el siguiente grafo:

```
1 //      0
2 //      / \
3 //      1  2
4 //      / \
5 //      3  4
```

La lista de adyacencia puede ser:

```
1 adj[0] = {1, 2};
2 adj[1] = {0, 3, 4};
3 adj[2] = {0};
4 adj[3] = {1};
5 adj[4] = {1};
```

Si comenzamos DFS desde 0, la pila inicialmente contiene:

```
1 st = {0}
```

Se extrae 0 y se agregan sus vecinos:

```
1 st = {1, 2}
```

El siguiente vértice que se extrae depende del orden en que fueron agregados a la pila. Por ejemplo, si se extrae primero 2, se procesa esa rama antes de regresar a 1.

DFS en un grafo desconectado:

Al igual que BFS, una sola ejecución de DFS solamente visita los vértices alcanzables desde el origen. Para recorrer todo el grafo, debemos iniciar un nuevo DFS desde cada vértice que todavía no haya sido visitado.

```
1 vector<bool> visitado(n, false);
2
3 for (int u = 0; u < n; u++) {
4
5     if (visitado[u]) continue;
6
7     dfs(u, adj, visitado);
8 }
```

Este patrón permite procesar todas las componentes conexas del grafo.

Contar componentes conexas:

Podemos contar el número de componentes incrementando un contador cada vez que sea necesario iniciar un nuevo DFS.

```
1 int componentes = 0;
2
3 vector<bool> visitado(n, false);
4
5 for (int u = 0; u < n; u++) {
6
7     if (visitado[u]) continue;
8
9     componentes++;
10    dfs(u, adj, visitado);
11 }
12
13 cout << componentes << '\n';
```

DFS en árboles:

En un árbol no es necesario mantener un arreglo de visitados si se conoce el padre del vértice actual. Al recorrer un vecino, simplemente se evita regresar al padre.

Sin embargo, en un grafo general se recomienda utilizar **visitado**, ya que pueden existir ciclos.

```
1 void dfs(int inicio, vector<vector<int>>& adj) {
2
3     stack<pair<int, int>> st;
4
5     st.push({inicio, -1});
6
7     while (!st.empty()) {
8
9         auto [u, padre] = st.top();
10        st.pop();
11
12        cout << u << ' ';
13
14        for (int v : adj[u]) {
15
16            if (v == padre) continue;
17
18            st.push({v, u});
19        }
20    }
21 }
```

Esta última versión es apropiada para árboles, pero para grafos generales debe utilizarse un arreglo de visitados.

Aplicaciones:

DFS es una de las herramientas fundamentales para resolver problemas de grafos. Entre sus aplicaciones más comunes se encuentran:

- Recorrer todos los vértices alcanzables desde un origen.
- Encontrar componentes conexas.

- Detectar ciclos.
- Construir árboles DFS.
- Encontrar puntos de articulación y puentes.
- Realizar ordenamiento topológico.
- Resolver problemas de conectividad.
- Recorrer árboles y estructuras jerárquicas.

Complejidad:

Utilizando una lista de adyacencia, cada vértice se procesa una sola vez y cada arista se revisa una cantidad constante de veces. Por lo tanto:

$$O(n + m)$$

La pila y el arreglo de visitados requieren $O(n)$ memoria adicional.

Representación	Tiempo	Memoria adicional
Lista de adyacencia	$O(n + m)$	$O(n)$
Matriz de adyacencia	$O(n^2)$	$O(n)$

Table 16: Complejidad de DFS según la representación del grafo.

Nota: DFS y BFS tienen la misma complejidad asintótica $O(n + m)$ utilizando una lista de adyacencia, pero exploran el grafo de manera diferente. BFS procesa por niveles mediante una cola, mientras que DFS profundiza utilizando una pila.

Regla práctica: utiliza DFS cuando el problema dependa de explorar profundamente una estructura, recorrer componentes, detectar ciclos o realizar algún procesamiento basado en el orden de entrada y salida de los vértices. Si el problema requiere distancias mínimas en un grafo no ponderado, normalmente BFS es la opción adecuada.

6.6 DFS para Conteo de Caminos

6.7 DFS + Cerramiento Transitivo

6.8 Flood Fill

6.9 Laberinto con BFS

6.10 Tour de Caballo de Ajedrez

6.11 Mínimo Número de Movimientos de Caballo

6.12 Serpientes y Escaleras

7 Conectividad

7.1 Componentes Conexas

7.2 Detección de Ciclos

7.3 Ciclo en Grafo Dirigido

7.4 Puentes

7.5 Puntos de Articulación

7.6 Componentes Biconectados

7.7 Tarjan

8 Topological Sort

8.1 Kahn's Algorithm

8.2 DFS + Orden Topológico

8.3 Detección de Ciclos con Topological Sort

9 Single Source Shortest Path (SSSP)

9.1 BFS para Caminos Más Cortos

9.2 0-1 BFS

9.3 Dijkstra

9.4 Bellman-Ford

9.5 Shortest Path en Grafos Binarios

9.6 Shortest Path desde Múltiples Orígenes

9.7 Shortest Path en Grafos Multietapa

9.8 Shortest Path con Exactamente k Aristas

10 All Pairs Shortest Path (APSP)

10.1 Floyd-Warshall

11 Minimum Spanning Tree (MST)

11.1 Disjoint Set Union (DSU)

11.2 Kruskal

11.3 Prim

12 Strongly Connected Components (SCC)

12.1 Kosaraju

12.2 Tarjan SCC

12.3 Condensation Graph

13 Bipartite Graphs

13.1 2-Coloring

13.2 Bipartite Matching

14 Trees

14.1 DFS en Árboles

14.2 BFS en Árboles

14.3 Diámetro de un Árbol

14.4 Centro de un Árbol

14.5 Combinatoria en Árboles Ordenados

14.6 Tree DP

14.7 Rerooting

15 Lowest Common Ancestor (LCA)

15.1 Binary Lifting

15.2 Euler Tour + RMQ

15.3 Distancia entre Dos Nodos

15.4 k -ésimo Ancestro

16 Tree Queries

16.1 Euler Tour

16.2 Subtree Queries

16.3 Path Queries

16.4 Heavy-Light Decomposition

16.5 Heavy-Light Decomposition + Fenwick

16.6 Heavy-Light Decomposition + Segment Tree

17 Disjoint Set Union

17.1 Union-Find

17.2 Path Compression

17.3 Union by Size / Rank

17.4 Unión de Conjuntos Disjuntos en un Árbol

17.5 DSU Offline

17.6 DSU Rollback

18 Graph Decomposition

18.1 Bridge Tree

18.2 Block-Cut Tree

18.3 Condensation Graph

19 Eulerian Graphs

19.1 Eulerian Path

19.2 Eulerian Circuit

19.3 Hierholzer

20 Network Flow

20.1 Ford-Fulkerson

20.2 Edmonds-Karp

20.3 Dinic

20.4 Min-Cut

20.5 Maximum Bipartite Matching

21 2-SAT

21.1 Implication Graph

21.2 2-SAT con SCC

22 Backtracking y Problemas Clásicos

22.1 Problema de las N Reinas

22.2 Variaciones del Problema de las N Reinas

22.3 Tour de Caballo

23 Advanced Graph Algorithms

23.1 Centroid Decomposition

23.2 DSU on Tree

23.3 Hopcroft-Karp

23.4 Min-Cost Max-Flow

23.5 DSU Rollback

Parte IV Ordenamiento y Búsqueda

24 Ordenamiento y Búsqueda (Sorting & Searching)

Varias búsquedas lineales y sub-lineales. (La búsqueda binaria tiene su propia sección.)

24.1 Número faltante (Missing Number)

Idea: los números $0, 1, \dots, n$ deberían sumar $\frac{n(n+1)}{2}$ (fórmula de Gauss). Si falta uno, la suma real queda corta *exactamente* en ese valor, así que el faltante es (suma esperada – suma real). Basta un recorrido para acumular la suma. Alternativa sin riesgo de overflow: hacer XOR de $0..n$ y de todos los elementos; lo que quede es el faltante. **Complejidad:** $O(n)$.

```
1 long long esp = (long long)n*(n+1)/2; // suma 0..n
2 long long falta = esp - accumulate(a.begin(), a.end(), 0LL);
```

24.2 Máximo / Mínimo (Array Max/Min)

Idea: recorres el arreglo una sola vez llevando el menor y el mayor vistos hasta ahora; comparas cada elemento contra ambos y actualizas. No requiere ordenar (ordenar sería $O(n \log n)$, peor). Con un recorrido consigues los dos a la vez. **Complejidad:** $O(n)$.

```
1 int mn = *min_element(a.begin(), a.end());
2 int mx = *max_element(a.begin(), a.end());
```

24.3 Par de suma absoluta mínima

Idea: buscamos el par cuya suma esté *más cerca de cero*. Tras ordenar, ponemos un puntero en cada extremo. La suma de los extremos te dice hacia dónde moverte: si es negativa, para acercarla a 0 hay que subir el menor (`lo++`); si es positiva, hay que bajar el mayor (`hi--`). Cada movimiento descarta el candidato menos prometedor, y en el camino guardas el $|suma|$ más pequeño visto. Los dos punteros recorren el arreglo una vez ($O(n)$); domina el sort. **Complejidad:** $O(n \log n)$.

```
1 sort(a.begin(), a.end());
2 int lo=0, hi=a.size()-1; long long mej=LLONG_MAX;
3 while(lo<hi){ long long s=(long long)a[lo]+a[hi];
4     mej=min(mej, llabs(s));
5     if(s<0) lo++; else hi--; }
```


24.4 Par con diferencia dada (Difference Pair)

Idea: ¿existe un par cuya diferencia sea exactamente d ? Tras ordenar, dos punteros avanzan hacia adelante: si la diferencia actual $a_j - a_i$ es menor que d , adelantas el puntero lejano (j) para agrandarla; si es mayor, adelantas el cercano (i) para achicarla; si es igual, encontraste el par. Como el arreglo está ordenado, la diferencia crece con j y decrece con i , así que el barrido nunca retrocede. **Complejidad:** $O(n \log n)$ (domina el sort).

```
1 sort(a.begin(), a.end()); int i=0, j=1, n=a.size();
2 while(i<n && j<n){
3     if(i!=j && a[j]-a[i]==d) return true;
4     if(a[j]-a[i]<d) j++; else i++; }
```

24.5 Búsqueda ternaria (Ternary Search)

Idea: sirve para hallar el pico de una función **unimodal** (sube y luego baja, con un solo máximo — no para buscar un valor en un arreglo cualquiera). Partes el rango en tres con dos puntos $m_1 < m_2$ y comparas: si $f(m_1) < f(m_2)$, el máximo no puede estar a la izquierda de m_1 , así que descartas ese tercio; en caso contrario descartas el tercio derecho. Cada paso elimina $\sim \frac{1}{3}$ del rango, por eso converge en logarítmico. (Para un mínimo, invierte la comparación.) **Complejidad:** $O(\log n)$ — NO $O(n \log n)$.

```
1 while(hi-lo>2){ long long m1=lo+(hi-lo)/3, m2=hi-(hi-lo)/3;
2     if(f(m1)<f(m2)) lo=m1+1; else hi=m2-1; }
3 // revisa [lo,hi] y devuelve el x que maximiza f
```

24.6 Búsqueda de Fibonacci (Fibonacci Search)

Idea: misma estrategia que la binaria (descartar mitades de un arreglo ordenado), pero en vez de partir por el punto medio elige el punto de comparación usando números de Fibonacci consecutivos. Mantiene tres Fibonacci y en cada paso mira la posición $\text{off}+f_2$; según el elemento sea menor o mayor que el objetivo, retrocede uno o dos pasos en la secuencia. Ventaja: solo usa *sumas y restas*, nunca división, lo que la hace útil en hardware sin división rápida o para saltar por posiciones amigables con la caché. **Complejidad:** $O(\log n)$.

```
1 int f2=0, f1=1, fb=f1+f2;
2 while(fb<n){ f2=f1; f1=fb; fb=f1+f2; }
3 int off=-1;
4 while(fb>1){ int i=min(off+f2, n-1);
5     if(a[i]<x){ fb=f1; f1=f2; f2=fb-f1; off=i; }
6     else if(a[i]>x){ fb=f2; f1=f1-f2; f2=fb-f1; }
7     else return i; }
8 if(f1==1 && off+1<n && a[off+1]==x) return off+1;
```

(En Java es idéntico al C++, cambiando `a.length`.)

24.7 Búsqueda exponencial (Exponential Search)

Idea: dos fases sobre un arreglo **ordenado**. Primero *acotas* el objetivo doblando el índice (1, 2, 4, 8, ...) hasta que $a[i]$ lo supera; en ese momento sabes que el objetivo está entre $i/2$ e i . Segundo, haces una binaria normal dentro de ese bloque. Como el bloque encontrado tiene tamaño proporcional a la posición del objetivo, es ideal cuando el arreglo es enorme o de tamaño desconocido y el objetivo está cerca del inicio. **Complejidad:** $O(\log n)$.

```
1 if(a[0]==x) return 0;
2 int i=1; while(i<n && a[i]<=x) i*=2; // dobla el rango
3 // luego binaria en [i/2, min(i,n-1)]
```

24.8 Búsqueda por saltos (Jump Search)

Idea: en vez de revisar elemento por elemento, avanzas en bloques de tamaño $\sqrt{n}$ mirando solo el *último* de cada bloque; cuando ese último ya supera al objetivo, sabes que (si existe) está en el bloque anterior, así que lo barres linealmente. El balance $\sqrt{n}$ saltos + $\sqrt{n}$ del barrido minimiza el total. Punto medio entre la lineal ($O(n)$) y la binaria ($O(\log n)$), útil cuando saltar hacia atrás es costoso. **Complejidad:** $O(\sqrt{n})$.

```
1 int paso=max(1, (int)sqrt((double)n)), prev=0;
2 while(prev<n && a[min(prev+paso, n)-1]<x) prev+=paso;
3 for(int i=prev; i<min(prev+paso, n); i++)
4     if(a[i]==x) return i;
```

24.9 Heap Sort

Idea: un *heap* binario se representa en el mismo arreglo: el hijo izquierdo de i es $2i+1$ y el derecho $2i+2$; en un *max-heap* todo padre es $\geq$ sus hijos, así que el mayor está en la raíz. Heap Sort tiene dos fases. **(1) Construir el heap:** recorres de abajo hacia arriba aplicando *bajar* (sift-down) a cada padre. Aunque parezca $O(n \log n)$, es $O(n)$: casi todos los nodos están cerca de las hojas y se hunden poco (la suma $\sum n/2^h \cdot h$ converge a $2n$). **(2) Ordenar:** repetidamente intercambias la raíz (el mayor) con el último elemento activo —quedando ya ordenado al final— y “hundes” la nueva raíz para restaurar el heap; cada una de las n extracciones cuesta $O(\log n)$. In-place, sin memoria extra, pero **no estable**. **Complejidad:** $O(n \log n)$ en todos los casos.

```
1 void bajar(vector<int>&a, int n, int i){ // hunde: O(log n)
2     while(true){ int m=i, l=2*i+1, r=2*i+2;
3         if(l<n && a[l]>a[m]) m=l;
4         if(r<n && a[r]>a[m]) m=r;
5         if(m==i) break; swap(a[i], a[m]); i=m; } }
6 void heapSort(vector<int>&a){ int n=a.size();
7     for(int i=n/2-1; i>0; i--) bajar(a, n, i); // heap: O(n)
8     for(int i=n-1; i>0; i--){ swap(a[0], a[i]); // max al final
9         bajar(a, i, 0); } }
```

24.10 QuickSelect

Idea: encuentra el k -ésimo menor *sin ordenar todo*. Como quicksort, particiona alrededor de un pivote dejando a la izquierda lo menor y a la derecha lo mayor; pero como sabes en qué lado cae la posición k , recorres (o iteras) sobre *un solo* lado y descartas el otro. Por eso el promedio baja de $O(n \log n)$ a $O(n)$: cada fase procesa la *mitad* del anterior, y $n + \frac{n}{2} + \frac{n}{4} + \dots = 2n$. El peor caso $O(n^2)$ aparece con pivotes pésimos (arreglo casi ordenado + pivote extremo); **elegir el pivote al azar** lo vuelve prácticamente imposible. La versión de abajo es iterativa (sin pila de recursión). Si necesitas garantía $O(n)$ en el peor caso, existe *mediana de medianas* para elegir un buen pivote. **Complejidad:** promedio $O(n)$, peor $O(n^2)$.

```
1 int quickSelect(vector<int> a, int k){ // k in [0, n-1]
2     int lo=0, hi=a.size()-1;
3     while(lo<hi){
4         int p=a[lo+rand()%(hi-lo+1)], i=lo, j=hi; // pivote aleatorio
5         while(i<=j){ while(a[i]<p) i++; while(a[j]>p) j--;
6             if(i<=j){ swap(a[i], a[j]); i++; j--; } }
7         if(k<=j) hi=j; // k a la izq
8         else if(k>=i) lo=i; // k a la der
9         else return a[k];
10    }
11    return a[lo];
12 }
```

```
13 // C++ trae nth_element(a.begin(), a.begin()+k, a.end()); // O(n)
```

(En Java igual, con `Random`; código completo en el repo.)

25 Búsqueda Binaria (Binary Search)

Idea central: mantienes un rango $[lo, hi]$ donde *sabes* que está la respuesta y en cada paso lo partes por la mitad, descartando la mitad que no puede contenerla. Como reduces a la mitad cada vez, llegas en $\log_2 n$ pasos. Funciona si los datos están **ordenados** o, más general, si existe un **predicado monótono** (una condición que pasa de falsa a verdadera una sola vez): entonces buscas la frontera del cambio. **Complejidad:** $O(\log n)$ en discreto, $O(\text{iter})$ en reales, $O(L \log n)$ con cadenas. Tip: usa $lo + (hi - lo) / 2$ para el `mid` y evitar overflow.

25.1 En arreglo ordenado

Cómo: comparas x con el elemento del medio; si es menor, descartas la mitad derecha, si es mayor la izquierda. `lower_bound` te da el primer índice con valor $\geq x$ y `upper_bound` el primer $> x$; con eso sabes si x existe y cuántas copias hay.

```
1 int i = lower_bound(a.begin(), a.end(), x) - a.begin(); // >= x
2 int j = upper_bound(a.begin(), a.end(), x) - a.begin(); // > x
3 bool hay = binary_search(a.begin(), a.end(), x);
```

25.2 Menor x que cumple

Cómo: aquí no buscas en un arreglo sino en el *rango de respuestas posibles*. El predicado es monótono F.F T..T (una vez que algo “sirve”, todo lo mayor también). En cada paso pruebas el `mid`: si cumple, lo guardas como candidato y sigues buscando a la izquierda ($hi = mid - 1$) por si hay uno menor que también sirva; si no cumple, subes ($lo = mid + 1$). Al final `res` es la frontera izquierda: el *menor* que cumple. Ejemplo: “mínima capacidad para repartir en $\leq k$ días”.

```
1 long long res=-1;
2 while(lo<=hi){ long long mid=lo+(hi-lo)/2;
3   if(cumple(mid)){res=mid; hi=mid-1;} // sirve -> izq
4   else lo=mid+1; }
```

25.3 Mayor x que cumple

Cómo: el caso espejo. Ahora el predicado es T..T F..F (sirve hasta cierto punto y luego deja de servir). Cuando el `mid` cumple, lo guardas y buscas a la *derecha* ($lo = mid + 1$) por si hay uno mayor que también sirva; si no cumple, bajas ($hi = mid - 1$). Respecto al anterior solo cambian esas dos líneas. Ejemplo: “máxima longitud de corte tal que salgan $\geq k$ piezas”.

```
1 long long res=-1;
2 while(lo<=hi){ long long mid=lo+(hi-lo)/2;
3   if(cumple(mid)){res=mid; lo=mid+1;} // sirve -> der
4   else hi=mid-1; }
```

25.4 Sobre reales

Cómo: cuando la respuesta es un número *real* (raíces, puntos de equilibrio) no hay “mitad entera” ni condición $lo \leq hi$ para parar. En su lugar iteras un número fijo de veces (unas 100 basta: el rango se divide por 2^{100} , precisión más que suficiente), moviendo `lo` o `hi` al `mid` según el predicado. Al terminar, $lo \approx hi$ es la frontera buscada.

```
1 for(int it=0; it<100; it++){ double mid=(lo+hi)/2;
2   if(cumple(mid)) hi=mid; else lo=mid; }
3 // lo - hi = frontera
```

25.5 Con strings

Cómo: exactamente la misma binaria, pero comparando cadenas en orden *lexicográfico* (diccionario) en lugar de números. La única diferencia de costo es que comparar dos cadenas cuesta $O(L)$ (longitud), no $O(1)$, así que el total sube a $O(L \log n)$. Ojo: el arreglo debe estar ordenado con el mismo criterio (ASCII: mayúsculas antes que minúsculas).

```
1 // vector<string> s ordenado
2 int i = lower_bound(s.begin(), s.end(), string("caro"))
3   - s.begin();
4 bool hay = binary_search(s.begin(), s.end(), string("caro"));
```

Parte V Matemáticas

26 Matemáticas (Math)

Contenido en desarrollo.

27 Teoría de Números (Number Theory)

Contenido en desarrollo.

Contenido en desarrollo.

Parte VI Geometría Computacional

Parte VII Programación Dinámica (Dynamic Programming)

Contenido en desarrollo.

Parte VIII Algoritmos Voraces (Greedy)

Contenido en desarrollo.

Parte IX Ad-hoc y Simulación

Contenido en desarrollo.

Parte X String Processing

28 Cadenas (Strings)

Contenido en desarrollo.

Parte XI BitWise

Parte XII Game Theory